MQTT Version 6.0 — Industrial Stream & Advanced Queuing Extension

Bill Sommers, AI Solutions Lead, HiveMQ, Inc.

Why MQTT v6.0? Motivation, Rationale, and Intended Audience

> ***Status:*** *Part of the MQTT v6.0 Draft Proposal — March 2026*

---

1. Who This Is For

MQTT v6.0 is targeted at a specific tier of industrial and enterprise IoT deployments that have **outgrown what MQTT v5.0 was designed to provide**. It is not a general-purpose upgrade for all MQTT users.

Primary Audience

Audience	Why They Need v6.0
Large-scale industrial operators — energy utilities, grid SCADA, semiconductor fabs, process manufacturing	Need ordered, durable message queues that survive broker restarts and client disconnections; need competing consumer patterns for high-availability pipelines
IIoT platform engineers	Building middleware that must guarantee message delivery ordering across distributed broker clusters without application-layer workarounds
Data pipeline architects	Connecting MQTT edge data to downstream analytics, historians, and SCADA systems; need pull-based flow control so consumers can dictate their own ingestion rate
HiveMQ extension developers	Implementing advanced queuing semantics today via custom extensions; v6.0 standardises and formalises these patterns
MQTT broker implementers	Seeking a standardised wire protocol for durable queuing so customers are not locked into proprietary extension APIs

Who Should Stay on MQTT v5.0

If your deployment matches any of the following profiles, v5.0 is the right choice and v6.0 adds complexity without benefit:

- Simple telemetry from sensors to a single backend consumer
- Devices with less than 256 KB available memory
- Short message lifetimes where loss is acceptable (dashboards, live monitoring)
- No requirement for competing consumers or consumer group semantics
- Broker infrastructure managed by a single operator with no cluster failover requirements

---

2. The Problem Statement: What MQTT v5.0 Cannot Do

MQTT v5.0 is a well-designed protocol for lightweight publish-subscribe messaging. It was not designed to be a message queue. Five specific limitations emerge at scale in industrial deployments:

2.1 No Durable, Session-Independent Queues

MQTT v5.0 sessions are bound to a client identifier and a broker node. When a client disconnects: - Clean session (`cleanStart=true`): all queued messages are discarded - Persistent session (`cleanStart=false`): messages are held *for that specific client*, not for a named consumer group

The result: If a downstream analytics pipeline (one of ten competing consumers) disconnects for maintenance, its share of the message backlog is either lost or held indefinitely in a single-consumer session — neither behaviour is correct for an industrial pipeline.

What operators do today: They build application-layer queuing on top of MQTT using Redis, Kafka, or proprietary HiveMQ extensions. The MQTT broker becomes a thin transport layer and the real queuing logic lives outside the protocol. This defeats the purpose of using a standardised protocol.

2.2 No Message Ordering Guarantee Across Sessions

MQTT v5.0 uses a 16-bit Packet Identifier that resets per session. Messages across sessions cannot be ordered. If a consumer reconnects mid-stream, it cannot determine which messages it has already processed and which it has not — not without an application-layer sequence number that the publisher must attach manually.

The result: Exactly-once processing at the application layer requires every publisher and consumer pair to implement bespoke deduplication logic. In a system with 50 publishers and 20 consumers, this is 1,000 pairwise agreements — none of them standardised.

2.3 No Pull-Based Flow Control

MQTT v5.0 is push-only. The broker delivers messages to subscribers as fast as they arrive. Consumer-side flow control is limited to: - `Receive Maximum` — caps the in-flight window but does not let the consumer control *when* it receives the next batch - Disconnecting and reconnecting — a destructive and operationally expensive throttle

The result: A slow consumer — one doing a database write per message — is flooded. The broker's delivery queue fills the consumer's memory until it crashes. Consumer crashes trigger reconnects, which trigger session resumption, which floods the consumer again. This cycle is well-known in industrial MQTT deployments and is solved today by external rate limiters, message buffers, or complete replacement of the push model with polling.

2.4 No Cluster-Aware Sequence Numbers

MQTT v5.0 has no concept of cluster topology. If a client is connected to Node A and Node A fails, the client reconnects to Node B. From the protocol's perspective, this is a new session — there is no way to tell Node B "I had processed up to message X on Node A; please resume from X+1."

The result: After any broker failover, consumers must either: - Re-process all messages from the beginning of the session (duplicate processing) - Trust that the application layer tracked its own position (requires external state) - Accept a gap (message loss)

None of these outcomes are acceptable for regulatory data, safety-critical commands, or financial telemetry.

2.5 Shared Subscriptions Have Loose Delivery Semantics

MQTT v5.0 introduced Shared Subscriptions (`$share/`) for competing consumers. However: - There is no guaranteed round-robin; delivery order within a group is implementation-defined - There is no concept of exclusive access (hot-standby primary/backup) - If a consumer in the group disconnects mid-processing, the in-flight message may be re-delivered to any other consumer — but only if the broker's session timeout has not expired

The result: Building a reliable competing consumer pipeline on `$share/` requires extensive application-layer coordination that must be re-implemented for each new deployment.

---

3. Why Not Use Kafka or AMQP?

This is the most frequently raised objection. The answer depends on **where in the architecture** the problem exists.

At the Edge (Sensors, RTUs, PLCs, Smart Meters)

Kafka and AMQP are not viable at the edge: - Kafka's minimum client footprint is ~10 MB — not compatible with microcontrollers - AMQP 1.0 is a complex framing protocol — correct implementation requires significant expertise and is poorly suited to constrained devices - Kafka requires TCP connections with persistent state; many OT networks have unreliable connectivity and require disconnect-tolerant protocols - MQTT v5.0 is already deployed on hundreds of millions of devices with battle-tested client libraries for C, MicroPython, Java, and Go

Conclusion: At the edge, MQTT remains the right protocol. The question is what happens to messages *after* they reach the broker cluster.

At the Broker Tier (The Gap MQTT v6.0 Fills)

Today, industrial deployments solve the queuing problem with a pattern like this:

```
[Edge device MQTT v5.0] → [MQTT Broker] → [Bridge to Kafka/AMQP] → [Consumer]
```

This architecture requires: - A bridge component that must be deployed, scaled, and maintained - Two separate protocol stacks, two separate failure domains - Message translation overhead at the bridge - Kafka or AMQP client libraries in every downstream consumer

MQTT v6.0 eliminates the bridge by extending the MQTT broker to natively support the queuing semantics that consumers need. The edge devices continue to speak MQTT v5.0 unchanged. The downstream consumers get ordered, pull-based, durable message queues — without a separate message queue system.

```
[Edge device MQTT v5.0] → [HiveMQ v6.0 Broker — native queuing] → [Consumer via FETCH]
```

This is not "MQTT trying to become Kafka." It is MQTT extending its native durability semantics to the broker tier where the queuing problem actually lives.

---

4. Design Principles

MQTT v6.0 is built on five explicit principles, each derived from a lesson learned in large-scale industrial deployments:

Principle 1: The Protocol Must Survive Broker Restarts

A `$queue/` message that has been published MUST be retained by the broker cluster regardless of individual node failures. The protocol defines what "retained" means (quorum write), when sequence continuity can be guaranteed (same Epoch), and when it cannot (Epoch increment on counter quorum loss).

This is a departure from v5.0's "retained message" feature, which is per-topic and not queue-semantics aware.

Principle 2: Consumers Control Their Own Rate

Pull-based FETCH (not push) is the default for `$queue/` consumption. A consumer that is slow, restarting, or catching up a backlog is never flooded. It fetches when it is ready.

Push subscriptions remain available for compatibility and for use cases where push semantics are correct (live telemetry monitoring, real-time dashboards).

Principle 3: Sequence Numbers Are a First-Class Protocol Citizen

Every `$queue/` message has a broker-assigned, cluster-wide monotonic sequence number (Property `0x30`). Publishers do not assign sequence numbers — the broker does, atomically, with no routing constraints. Sequence numbers are: - Immutable once assigned - Cluster-wide (not per-session or per-node) - The basis for exactly-once deduplication at the consumer - The basis for gap detection and recovery

Principle 4: Cluster Failover Must Be Transparent to Well-Behaved Clients

A client that stores its `last-seq` and `epoch` values can reconnect to *any* node in the cluster after any failure and resume processing from exactly where it left off — provided the cluster maintained quorum. If quorum was lost (an Epoch increment event), the protocol signals this explicitly so the client can decide how to recover.

No application-layer failover logic is required. No broker-specific reconnect APIs. The protocol carries all the state the client needs.

Principle 5: Backward Compatibility Is a Non-Negotiable Constraint

MQTT v6.0 is designed so that: - A v6.0 broker can serve v5.0 and v3.1.1 clients without modification - A v5.0 broker can serve v6.0 clients in compatibility mode (via User Properties and Virtual FETCH) - The migration from v5.0 to v6.0 does not require a flag-day cutover

The compatibility layer is specified precisely enough that it can be implemented as a HiveMQ Extension Plugin without modifying the core broker.

---

5. Expected Benefits

For Industrial Operators

Benefit	Mechanism
Message loss eliminated for `$queue/` topics	Quorum write + HWM-based exactly-once
Consumer restart is safe — no re-processing required	`last-seq` + `epoch` in CONNECT; broker resumes from N+1
Slow consumers cannot be flooded	Pull-based FETCH; consumer controls batch size and timing
Competing consumers with correct failover	SQMC competing mode: in-flight locks re-dispatched on consumer crash
Hot-standby consumer for safety-critical commands	SQMC exclusive mode: single active consumer with automatic promotion
Broker failover transparent to clients	Epoch-based resync protocol

For Platform Engineers

Benefit	Mechanism
Eliminates MQTT-to-Kafka bridge for queue use cases	Native `$queue/` semantics in the broker
Standardised competing consumer semantics	SQMC spec replaces bespoke `$share/` orchestration
Sequence numbers enable cross-system correlation	`Seq` + `Epoch` = immutable event identifier for audit, replay, join
No external state required for exactly-once	HWM tracked by broker + client; no Redis or database required

For the MQTT Ecosystem

Benefit	Mechanism
Clear upgrade path from v5.0	Compatibility layer; no flag-day migration
Standardised semantics reduce broker lock-in	Wire protocol defines queue behaviour, not just transport
Opens MQTT to new use cases	Financial telemetry, ERP integration, command-and-control audit trails

---

6. Explicit Non-Goals

MQTT v6.0 explicitly does not aim to:

- Replace Kafka or RabbitMQ for server-to-server batch processing at data centre scale
- Provide full AMQP 1.0 semantics (routing topologies, exchanges, dead-letter queues)
- Support messages larger than the current MQTT maximum (256 MB per PUBLISH)
- Implement stream processing or CEP (complex event processing)
- Define a management API or REST interface for broker administration
- Mandate a specific cluster topology or replication protocol (the spec defines *what* must be guaranteed, not *how*)

---

7. Relationship to the MQTT v5.0 Specification

MQTT v6.0 is a **strict superset of MQTT v5.0**. Every feature defined in the OASIS MQTT v5.0 specification remains valid and unchanged. The additions in v6.0 are:

1. **New packet type**: FETCH (Control Packet Type 16) — absent from v5.0
2. **New property identifiers**: `0x30` through `0x35`, `0x41` through `0x42` — reserved ranges not used by v5.0
3. **New topic namespace**: `$queue/` — conventionally reserved (similar to `$SYS/`) but now formally specified
4. **New connection negotiation**: Protocol Level `6` in CONNECT; `mqtt-ext: v6.0` User Property for compat mode
5. **New CONNECT fields**: `last-seq` and `epoch` for stateful reconnection

No existing v5.0 packet types, property identifiers, or semantics are modified or removed.

MQTT Version 6.0 — Industrial Stream & Advanced

Queuing Extension

> ***Status:*** *Draft Proposal — Based on exploratory design session, March 2026* ***Author:*** *Bill Sommers, AI Solutions Lead, HiveMQ, Inc.* ***Base Specification:*** *MQTT v5.0 OASIS Standard*

Table of Contents

Overview

1. Executive Summary
2. Motivation, Rationale & Intended Audience
3. Design Goals & Motivation

Specification

3. MQTT v6.0 Full Specification
4. Protocol Changes from v5.0
5. Compatibility Analysis

Architecture & Diagrams

6. System Architecture Diagrams
7. Packet Flow Diagrams
8. Cluster Failover & Epoch Diagrams

Analysis

9. Alternative Approaches
10. Critical Perspectives & Rebuttals
11. Technical Pitfalls & Mitigations
12. Performance Analysis: High-Volume & DEP Scale
13. Critical Analyses: Security, OT SRE, and Protocol Design

Reference Implementations

14. Python v6.0 Shim (`gmqtt`)
15. Protobuf Schema
16. HiveMQ Java Extension

Consolidated Reference

17. Full Proposal PDF (all sections + rendered diagrams)

Design Goals

MQTT v6.0 solves three hard problems of industrial IoT that MQTT v5.0 leaves to application-layer hacks:

Problem	v5.0 Status	v6.0 Solution
Message Ordering	16-bit per-session Packet ID, reused frequently	32-bit monotonic Stream Sequence (per queue, cluster-wide)

Problem	v5.0 Status	v6.0 Solution
Flow Control	Broker pushes all messages; consumer can be flooded	Pull-based FETCH packet; consumer dictates rate
Cluster Consistency	Opaque to clients; failover is implementation-defined	Epoch-based failover; clients detect and resync after cluster partition
Durable Queues	Session-bound; lost if client disconnects	Named `$queue/` namespace; persists independently of sessions
Consumer Patterns	Shared Subscriptions with loose semantics	SQMC: strict Competing (round-robin) and Exclusive (hot-standby) modes

---

Two Tracks

The proposal exists in **two variants** — see Alternatives for full discussion:

Track A: Native v6.0 (Breaking)

Introduces `FETCH` as a true Control Packet Type 16. Requires v6.0-aware broker and clients. Protocol Level in CONNECT is `6`.

Track B: Compatible Extension (Non-Breaking)

Tunnels all v6.0 features through v5.0 Property and User Property fields. Works on existing v5.0 brokers with a HiveMQ Extension Plugin. Protocol Level remains `5`.

The **recommended path** is Native v6.0 with an optional Compatibility Layer for hybrid deployments.

Key Terminology

Term	Definition
Stream Sequence Number	A 32-bit monotonic integer (Property `0x30`) assigned per `$queue/` message; immutable across the cluster
Stream Epoch	A 16-bit counter (Property `0x35`) incremented when cluster failover causes sequence discontinuity
FETCH	New Control Packet (Type 16) enabling pull-based flow control
`$queue/`	Topic namespace for named, durable, session-independent queues
SQMC	Single-Queue Multi-Consumer — strict load balancing with Competing or Exclusive semantics
Virtual FETCH	Compatibility-mode equivalent of FETCH using `PUBLISH` to `$SYS/queues/{name}/fetch`
High-Watermark	Client-side record of the last successfully processed sequence number
Distributed Sequence Counter	A cluster-wide atomic counter in HiveMQ's data grid, one per `$queue/`; any peer node increments it atomically to claim the next sequence number without routing constraints

---

Executive Summary: MQTT Version 6.0

What Is This?

MQTT v6.0 is a proposed evolutionary extension to the MQTT v5.0 protocol that upgrades it from a lightweight pub/sub message bus into an **industrial-grade stream messaging system**. It is designed to meet the reliability, ordering, and flow-control requirements of mission-critical environments — particularly semiconductor manufacturing (SECS/GEM), industrial IoT, and large-scale HiveMQ cluster deployments.

The Problem

MQTT v5.0 is excellent for lightweight telemetry and command delivery. However, it has structural limitations that force enterprise users to embed reliability logic inside application payloads:

- **No global message ordering.** The 16-bit Packet Identifier is per-session, transient, and recycled — making it impossible for a consumer to detect gaps caused by broker restarts or cluster failovers.
- **Push-only delivery.** The broker pushes all messages to consumers as fast as they arrive. A slow consumer cannot apply backpressure without complex out-of-band coordination.
- **Session-bound queues.** Messages are only held for a client while its session exists. There is no way to create a durable queue that outlives client connections.
- **Loose shared subscription semantics.** MQTT's `$share/` mechanism provides load balancing but lacks strict exclusive-consumer and message-locking semantics needed for transactional processing.
- **No cluster-aware consistency.** When a HiveMQ node fails and a client reconnects to a new node, the protocol has no mechanism to detect what was lost or resume from an exact point.

These gaps are currently "solved" by embedding sequence numbers, idempotency keys, and acknowledgment logic in message payloads — a non-standard, non-interoperable approach.

The Solution

MQTT v6.0 introduces five targeted additions to the protocol, all backward-compatible with existing v5.0 parsers:

1. 32-bit Stream Sequence Numbers

A new Property (`0x30`) carrying a monotonic, cluster-wide 32-bit integer is attached to every message in the `$queue/` namespace. This allows consumers to detect gaps (`received Seq 1002 but expected 1001`), perform deduplication, and implement application-level exactly-once processing without embedding logic in payloads.

2. Stream Epoch

A new Property (`0x35`) that tracks the "era" of a queue. When a HiveMQ cluster suffers a catastrophic failure and cannot guarantee sequence continuity, the Epoch is incremented. Consumers that detect an Epoch change know they must discard local idempotency state and re-synchronize — analogous to a SECS/GEM Establish Communications (S1F13) reset.

3. Named Durable Queues (`$queue/` namespace)

Topics prefixed with `$queue/` are treated as first-class queue entities: persisted to non-volatile storage, sequenced, and retained indefinitely until consumed or expired — independent of any client session. This replaces the implicit, session-bound queue model of v5.0.

4. Pull-Based Flow Control (FETCH)

A new `FETCH` Control Packet (Type 16) lets consumers explicitly request batches of messages from the broker. The broker holds messages in the persistent queue and releases them only when asked. This eliminates the "thundering herd" problem where a recovering consumer is flooded with backlogged messages. For v5.0-compat environments, a Virtual FETCH mechanism uses a `$SYS/` control topic instead.

5. Single-Queue Multi-Consumer (SQMC) Semantics

An extension to `SUBSCRIBE` that adds two new consumer modes beyond the loose `$share/` model: - **Competing:** Messages are distributed to exactly one consumer via round-robin, with strict message locking and immediate failover if the consumer disconnects before acknowledging. - **Exclusive:** One designated consumer receives all messages; others are hot standbys that take over instantly on failure, preserving strict ordering.

Business Impact

Use Case	v5.0 Risk	v6.0 Benefit
Semiconductor FAB (SECS/GEM)	Broker restart can silently lose a "Process Start" (S2F21) command, ruining a wafer	Epoch resync + sequence gaps ensure the command is detected as lost and retried
Industrial FDC Data Collection	High-frequency sensor data floods the host system, causing crashes	FETCH-based backpressure keeps the broker as the buffer; host processes at its own rate
Financial Message Bus	Duplicate processing of payment events due to lack of idempotency primitives	32-bit Sequence as an idempotency key, checked before executing business logic
Multi-node HiveMQ Cluster	Client failover to a new node silently re-delivers or loses messages	Last Received Seq in CONNECT allows new node to resume delivery from exact position
High-availability IoT Gateways	Primary/backup consumer logic must be hand-coded in application layer	Exclusive consumer mode with automatic hot-standby failover built into the protocol

---

Compatibility Strategy

v6.0 is designed to coexist with v5.0:

- All new properties use IDs in the reserved range (`0x30–0x45`) that v5.0 brokers and clients are required to ignore.
- The `$queue/` namespace gracefully degrades to standard topic behavior on v5.0 brokers (with appropriate ACL controls).
- A **Compatibility Layer** maps every native v6.0 feature to v5.0 User Properties and system topics, enabling mixed-version deployments.
- A negotiation handshake (`mqtt-ext: v6.0` in CONNECT/CONNACK) allows clients and brokers to detect v6.0 support and fall back gracefully.

Implementation Path for HiveMQ

The broker-side implementation uses the HiveMQ Extension SDK:

Feature	HiveMQ API
Sequence injection	`PublishInboundInterceptor`
Client throttling	`ClientService.blockClient()` / `unblockClient()`
Queue persistence	`ManagedPersistenceService`
Cluster epoch management	`ClusterService` + consistent hashing on queue names

A proof-of-concept can be built today using a HiveMQ Extension alongside a Python `gmqtt` client shim, without any changes to the core broker binary.

Recommendation

Adopt MQTT v6.0 as a **two-phase rollout**:

1. **Phase 1 — Compatible Extension:** Deploy the HiveMQ Extension Plugin that intercepts `v6-*` user properties. Migrate clients to the Python/Java shim. Zero broker downtime required.
2. **Phase 2 — Native v6.0:** Once the ecosystem supports Protocol Level 6, deprecate the shim layer and use native FETCH packets with full binary efficiency.

This proposal is production-ready as a HiveMQ extension today and provides a clear path to a formal OASIS standardization submission.

MQTT Version 6.0

Industrial Stream and Advanced Queuing Extension

OASIS Standard Draft Date: March 2026 **Author:** Bill Sommers, AI Solutions Lead, HiveMQ, Inc. **Base Specification:** MQTT Version 5.0 OASIS Standard (https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html)

---

1. Introduction

MQTT is a Client-Server publish/subscribe messaging transport protocol. It is lightweight, open, simple, and designed to be easy to implement.

[MODIFIED for v6.0]: Version 6.0 extends this simplicity with **Deterministic Stream Semantics**. These features provide the reliability required for mission-critical industrial environments — including semiconductor fabrication (SECS/GEM), financial messaging, and large-scale IoT deployments — where message ordering, pull-based flow control, and cluster-wide consistency are mandatory.

MQTT v6.0 is designed to be **wire-compatible** with MQTT v5.0 when the Compatibility Layer is active. All v6.0 extensions utilize the Property and User Property fields that v5.0 mandates be ignored when unrecognized. In Native Mode, v6.0 introduces a new Control Packet type and requires Protocol Level 6.

1.1 Conformance

A Client or Server is conformant to MQTT v6.0 if it:

- Meets all normative requirements of MQTT v5.0, AND
- Correctly handles Properties `0x30` (Stream Sequence Number) and `0x35` (Stream Epoch), AND
- Implements the `$queue/` namespace persistence semantics described in Section 4.1, AND
- Implements the v6.0 negotiation handshake described in Section 3.1.

1.2 Terminology

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in RFC 2119.

[NEW for v6.0] Additional Terms:

Term	Definition
Stream Sequence Number	A 32-bit monotonic integer uniquely identifying a message within a `$queue/` namespace, persistent across sessions
Stream Epoch	A 16-bit counter representing the “era” of a queue; incremented when cluster failover may have caused sequence discontinuity
FETCH	New Control Packet (Type 16) for pull-based message retrieval from persistent queues
`$queue/`	Topic namespace prefix designating a named, durable, session-independent persistent queue
SQMC	Single-Queue Multi-Consumer — a queuing pattern where each message is delivered to exactly one of multiple active consumers
Virtual FETCH	Compatibility-mode representation of FETCH using PUBLISH to `$SYS/queues/{name}/fetch`
High-Watermark	The last sequence number successfully processed by a consumer client
Distributed Sequence Counter	A cluster-wide atomic counter maintained in HiveMQ’s replication data grid, one per `$queue/`; any node may increment it atomically to claim the next sequence number
Competing Consumer	SQMC mode: messages distributed round-robin; each message delivered to exactly one consumer
Exclusive Consumer	SQMC mode: one designated consumer receives all messages; others are hot standbys

1.3 Normative References

- MQTT Version 5.0 OASIS Standard: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
- RFC 2119: Key words for use in RFCs to Indicate Requirement Levels

1.4 Data Representation

All data representation follows MQTT v5.0 conventions.

[NEW for v6.0]: - **Four Byte Integer:** A 32-bit unsigned integer in big-endian order. Used for Stream Sequence Numbers. - **Two Byte Integer:** A 16-bit unsigned integer in big-endian order. Used for Stream Epochs.

---

2. MQTT Control Packet Format

2.1 Fixed Header

Every MQTT Control Packet begins with a Fixed Header. The format is unchanged from v5.0.

```
Bit:     7    6    5    4    3    2    1    0
Byte 1:  [ MQTT Control Packet Type  ] [Flags]
Byte 2+: [        Remaining Length        ]
```

[NEW for v6.0] Control Packet Type 16 — FETCH:

Type Value	Name	Direction	Description
16	FETCH	Client to Server	Pull-based request for a batch of messages from a named queue

> ***Note on Compatibility:*** *The Type 16 slot was reserved in v5.0. A strict v5.0 broker will close the connection upon receiving a Type 16 packet. In environments with mixed v5.0/v6.0 brokers, use the Virtual FETCH mechanism (Section 4.2.1) instead.*

2.2 Variable Header Properties

MQTT v6.0 introduces four new Property Identifiers in the reserved range (`0x30–0x45`):

Hex	Name	Data Type	Applicable Packets
0x30	**Stream Sequence Number**	Four Byte Integer	PUBLISH, FETCH (response)
0x35	**Stream Epoch**	Two Byte Integer	PUBLISH, CONNACK, DISCONNECT
0x41	**Throughput Limit**	Four Byte Integer	CONNACK
0x42	**Batch Size**	Four Byte Integer	FETCH, PUBLISH (Virtual FETCH)

2.2.1 Stream Sequence Number (0x30)

- **Type:** Four Byte Integer (32-bit unsigned, big-endian)
- **Packets:** PUBLISH (for `$queue/` topics)
- **Normative:** The Broker MUST assign a monotonically increasing value to this property for every message published to a `$queue/` topic. If the Publisher provides this value, the Broker MUST validate it and MAY override it.
- **Purpose:** Provides a persistent, end-to-end message identity that survives transport-layer reconnections, session restarts, and cluster failovers. Unlike the 16-bit Packet Identifier (which is per-session and recycled), this value is globally unique within a queue's lifetime.

2.2.2 Stream Epoch (0x35)

- **Type:** Two Byte Integer (16-bit unsigned, big-endian)
- **Packets:** PUBLISH, CONNACK, DISCONNECT
- **Normative:** The Broker MUST include the current Epoch in CONNACK for resuming sessions. The Broker MUST increment the Epoch when it cannot guarantee sequence continuity after a cluster partition or replication quorum loss (see Section 4.3.2). HiveMQ does not use leader election; Epoch increments are driven by partition events, not node promotions. A Client that receives an Epoch value greater than its stored Epoch MUST discard its local idempotency table and perform a full application-layer resynchronization.
- **Purpose:** Identifies the "era" of a queue's sequence space. Allows clients to distinguish between a sequence gap (a missed message) and a cluster reset (all previous sequence context is invalid).

2.2.3 Throughput Limit (0x41)

- **Type:** Four Byte Integer (KB/second)
- **Packets:** CONNACK
- **Normative:** If present, the Client MUST NOT exceed this throughput. If the Client exceeds the limit, the Broker MAY disconnect with Reason Code `0x9B (Quota Exceeded)`.
- **Purpose:** Broker-enforced rate limiting for hardware NIC and storage protection.

2.2.4 Batch Size (0x42)

- **Type:** Four Byte Integer (message count)
- **Packets:** FETCH (native); PUBLISH to `$SYS/queues/{name}/fetch` (compat mode)
- **Normative:** The Broker MUST release at most this many messages in response to a FETCH or Virtual FETCH request. If fewer messages are available, the Broker MUST release only the available count.

---

3. MQTT Control Packets

3.1 CONNECT – Connection Request

3.1.2.2 Protocol Version

- **v5.0:** Protocol Level value is `5`
- **[MODIFIED for v6.0]:** Protocol Level value is `6` for native v6.0 mode. Clients MAY use Protocol Level `5` combined with the v6.0 User Property for Compatibility Mode operation.

3.1.2.11 CONNECT Properties

All v5.0 CONNECT properties are retained. The following property is added:

[NEW for v6.0] v6.0 Capability Negotiation: - **User Property Key:** `mqtt-ext` - **User Property Value:** `v6.0` - **Normative:** A Client wishing to use `$queue/` namespaces, FETCH semantics, or SQMC consumer modes MUST include this property. A Client that does not include it MUST be treated as a legacy v5.0 Client by the Broker.

[NEW for v6.0] Resync on Reconnect: - **User Property Key:** `v6-last-seq` - **User Property Value:** The decimal string representation of the last successfully processed Stream Sequence Number - **User Property Key:** `v6-epoch` - **User Property Value:** The decimal string representation of the Client's current known Epoch - **Normative:** Upon reconnect, a Client SHOULD include these properties to enable seamless queue resumption. The Broker MUST use these values to resume delivery from the correct position (see Section 4.3).

3.2 CONNACK – Connect Acknowledgement

3.2.2.3 CONNACK Properties

All v5.0 CONNACK properties are retained. The following properties are added:

[NEW for v6.0] v6.0 Capability Confirmation: - **User Property Key:** `mqtt-ext` - **User Property Value:** `v6.0` - **Normative:** A v6.0-aware Broker MUST echo this property. If absent, the Client MUST assume a legacy v5.0 Broker and MUST NOT use `$queue/` namespaces or FETCH semantics.

[NEW for v6.0] Throughput Limit (0x41): - See Section 2.2.3.

[NEW for v6.0] Stream Epoch (0x35): - **Normative:** REQUIRED when the Client included `v6-epoch` in CONNECT. If the Broker's current Epoch differs from the Client's reported Epoch, the Broker MUST include the new Epoch in CONNACK. The Client MUST respond per Section 2.2.2.

3.3 PUBLISH – Publish Message

3.3.2 PUBLISH Variable Header

The PUBLISH packet, when directed to a `$queue/` topic, triggers additional broker-side processing.

[MODIFIED for v6.0]: ~~The Server MUST push all matching messages to connected Clients.~~ For `$queue/` topics, the Server MUST NOT push messages unless a FETCH request is active or push mode is explicitly enabled. For standard topics, push behavior is unchanged.

3.3.2.3 PUBLISH Properties

All v5.0 PUBLISH properties are retained. The following properties are added:

[NEW for v6.0] Stream Sequence Number (0x30): - **Normative:** REQUIRED for all PUBLISH packets to `$queue/` topics. The Broker MUST assign this value before persisting the message. See Section 2.2.1.

[NEW for v6.0] Stream Epoch (0x35): - **Normative:** REQUIRED for all PUBLISH packets to `$queue/` topics. The Broker MUST include the current Epoch. See Section 2.2.2.

[NEW for v6.0] Priority: - **User Property Key:** `v6-priority` - **User Property Value:** Integer 0–255 (higher values = higher priority) - **Normative:** OPTIONAL. Higher-priority messages SHOULD be delivered to consumers before lower-priority messages in the same queue.

3.4 SUBSCRIBE – Subscribe Request

3.8.2.1 SUBSCRIBE Properties

All v5.0 SUBSCRIBE properties are retained. The following User Properties are added for SQMC semantics:

[NEW for v6.0] Consumer Semantics: - **User Property Key:** `v6-semantics` - **Values:** - `competing` — Enables Competing Consumer mode (round-robin; each message delivered to exactly one consumer; message locked until PUBACK received; immediate failover on disconnect) - `exclusive` — Enables Exclusive Consumer mode (primary consumer receives all messages; standby consumers receive nothing until primary disconnects) - **Normative:** If neither value is present, the subscription behaves as standard v5.0 Shared Subscription semantics (if `$share/` prefix is used) or normal subscription semantics otherwise.

[NEW for v6.0] Consumer Group: - **User Property Key:** `v6-group` - **User Property Value:** A UTF-8 string naming the consumer group - **Normative:** Clients in the same group sharing a `$queue/` topic with `competing` semantics will receive messages via round-robin distribution. Required when using `v6-semantics: competing`.

3.5 FETCH – Pull Request (Type 16) [NEW]

The FETCH packet is a new Control Packet introduced in v6.0. It enables pull-based message retrieval from persistent queues, providing absolute backpressure control.

3.5.1 FETCH Fixed Header

```
Bit:     7    6    5    4    3    2    1    0
Byte 1:  [ 1    0    0    0  ] [ 0    0    0    0 ]
          ^--- Type = 16 ---^   ^--- Reserved ---^
Byte 2+: [         Remaining Length              ]
```

3.5.2 FETCH Variable Header

Field	Size	Description
Packet Identifier	2 bytes	Standard acknowledgement identifier
Property Length	Variable	Length of property block
Batch Size (0x42)	4 bytes	Maximum messages to return (REQUIRED)
Wait Timeout (0x36)	2 bytes	Max ms to wait if queue is empty (long-poll; OPTIONAL)
Last Received Seq (0x30)	4 bytes	Client's high-watermark for gap detection (OPTIONAL)

3.5.3 FETCH Payload

The payload contains the **Queue Name** — a UTF-8 encoded string specifying the target `$queue/` topic (e.g., `$queue/production_line_1`).

```
+-------------------------------------------------------------+
|                  FIXED HEADER (2+ bytes)                    |
| Bits 7-4: Type (16) | Bits 3-0: Reserved (0000)           |
| Remaining Length (Variable Byte Integer)                    |
+-------------------------------------------------------------+
|                   VARIABLE HEADER                           |
| Packet Identifier       (2 bytes)                           |
| Property Length         (Variable)                          |
|   Batch Size    (0x42)  (4 bytes)                           |
|   Wait Timeout  (0x36)  (2 bytes, optional)                 |
|   Last Seq      (0x30)  (4 bytes, optional)                 |
+-------------------------------------------------------------+
|                       PAYLOAD                               |
| Topic Filter (UTF-8, e.g., "$queue/production_line_1")      |
+-------------------------------------------------------------+
```

3.5.4 FETCH Normative Behavior

- The Broker MUST respond by delivering at most `Batch Size` messages from the specified queue to the Client's session.
- If the queue contains fewer messages than `Batch Size`, the Broker MUST deliver only the available messages.
- If the queue is empty and `Wait Timeout` is specified, the Broker MAY hold the request open until a message arrives or the timeout expires (long-polling).
- If `Last Received Seq` is provided, the Broker MUST validate it against the queue's current state. If the Client's value is ahead of the Broker's known state, the Broker MUST respond with a DISCONNECT containing `Reason Code 0xA0 (Epoch Reset)`.
- The Broker MUST NOT send unsolicited messages from `$queue/` topics unless the Client has an active FETCH request.

3.5.5 FETCH Acknowledgement

The Broker responds to a FETCH by publishing messages with the standard PUBLISH flow (including PUBACK for QoS 1). The Packet Identifier from the FETCH is echoed in the response batch's PUBLISH packets to allow correlation.

---

4. Operational Behavior

4.1 The `$queue/` Namespace

Topics prefixed with `$queue/` represent **Named Queues** — durable, ordered message stores that exist independently of client sessions.

4.1.1 Persistence

- **Normative:** The Broker MUST store messages published to `$queue/` topics in non-volatile storage before acknowledging the PUBACK (QoS 1) or PUBCOMP (QoS 2).
- Messages in `$queue/` MUST NOT be discarded unless:
  - Acknowledged by all intended consumers (per SQMC consumer group logic), OR
  - The message's `Message Expiry Interval` (v5.0 Property 0x02) has elapsed, OR
  - The queue's storage capacity limit has been reached (see Section 4.1.3)

4.1.2 Ordering

- **Normative:** The Broker MUST deliver messages to consumers in strict ascending order of their `Stream Sequence Number`.
- The Broker MUST NOT deliver a message with Sequence N+1 before the consumer has acknowledged Sequence N (for QoS 1 and 2 delivery).

4.1.3 Storage Limits

- **User Property:** `v6-queue-max-size` in SUBSCRIBE — specifies the maximum number of messages to buffer (Four Byte Integer string)
- **Behavior on limit reached:** The Broker SHOULD apply a "drop-oldest" policy by default. An alternative "reject-new" policy returns PUBACK with Reason Code `0x8D (Quota Exceeded)`.
- **User Property:** `v6-queue-ttl` in SUBSCRIBE — specifies how long (seconds) the queue persists after the last consumer disconnects

4.1.4 Access Control

- **Normative:** Access to `$queue/` topics MUST be controlled via ACLs. The Broker SHOULD treat unauthorized publish or subscribe attempts to `$queue/` the same as unauthorized access to `$SYS/` topics.
- The Broker MUST reject subscribe requests to `$queue/` topics from Clients that have not completed the v6.0 negotiation handshake (Section 3.1).

4.2 Pull-Based Flow Control

MQTT v6.0 replaces the "push-all" delivery model for queue topics with a **consumer-driven pull model**.

4.2.1 Native Mode (FETCH Packet)

See Section 3.5. The Broker MUST hold queue messages until a FETCH request arrives. This provides absolute backpressure: if the consumer's application layer is slow, it simply does not issue FETCH requests.

4.2.2 Compatibility Mode (Virtual FETCH)

For deployments where clients or brokers do not support Protocol Level 6:

1. **Request:** The Client publishes a message (QoS 1) to `$SYS/queues/{queue_name}/fetch`
2. **Batch Size:** The PUBLISH MUST include `User Property: ("v6-batch", "{N}")` where N is the desired message count
3. **Response:** The Broker releases the next N messages from the persistent queue to the Client's session

4. **Normative:** The Broker MUST NOT release more than N messages per request. If no Virtual FETCH is active, the Broker MUST NOT push queue messages to the Client.

4.2.3 Rate Limiting

- **Throughput Limit (0x41) in CONNACK:** Broker-enforced KB/s limit on client publish rate
- **Reason Code 0x9B (Quota Exceeded):** Returned when a Client exceeds the throughput limit
- The `Receive Maximum` property from v5.0 continues to apply for QoS 1/2 in-flight limits

4.3 Cluster Consistency (Shared-Nothing Architecture)

HiveMQ is a **masterless, shared-nothing cluster**: all nodes are peers, there are no permanent leaders, and no consensus protocol (Raft/Paxos) is used. Any node can accept any publish from any client. v6.0 MUST preserve monotonic sequencing within this constraint using a **Distributed Sequence Counter** rather than a designated leader.

4.3.1 Distributed Sequence Counter

- **Normative:** Each `$queue/{name}` MUST maintain a single cluster-wide **Distributed Sequence Counter** stored in the Broker's replication data grid (e.g., HiveMQ's internal distributed data store).
- When a node receives a PUBLISH to a `$queue/` topic, it MUST perform an **atomic increment** (compare-and-swap) on that queue's counter to claim the next sequence number before persisting the message.
- No routing constraint is imposed on clients or publishers — **any node may sequence any publish**. Monotonicity is guaranteed by the atomic counter, not by sticky routing.
- If the atomic increment cannot complete because the counter's replication quorum is unavailable (e.g., a network partition has split the cluster), the receiving node MUST reject the PUBLISH with Reason Code `0x97 (Quota Exceeded)` and the client MUST retry after reconnection.
- The Broker MUST NOT assign a sequence number speculatively (i.e., without a confirmed atomic increment). Speculative assignment risks duplicate sequence numbers across nodes.

4.3.2 Epoch Management

- **Normative:** The Broker MUST increment the Stream Epoch when it cannot guarantee that the Distributed Sequence Counter's state survived intact. This occurs when:
  - A network partition resolves and the counter's quorum was lost, leaving the post-partition state uncertain; OR
  - One or more nodes that held the only replica of unacknowledged `$queue/` messages leave the cluster permanently.
- The new Epoch MUST be propagated in the next CONNACK or PUBLISH to any Client subscribed to the affected queue, and in a DISCONNECT to Clients that had in-flight messages at the time of the partition.
- Nodes rejoining after a partition MUST defer to the surviving cluster's Epoch value.

4.3.3 Client Failover and Resync

When a Client reconnects after a broker failure:

1. Client includes `v6-last-seq: {N}` and `v6-epoch: {E}` in CONNECT
2. Broker checks its local queue state for `$queue/{name}`:
   - If Epoch matches and Seq N is present: resume delivery from N+1
   - If Epoch matches but Seq N is absent (message was lost): begin from the earliest available message and flag the gap
   - If Epoch has incremented: respond with CONNACK containing the new Epoch value; Client MUST reset its idempotency state

4.3.4 Exactly-Once Processing (Application Layer)

Using the Stream Sequence Number as an idempotency key:

1. Consumer maintains a **High-Watermark** (last processed Seq) in an atomic store alongside its business logic
2. On message receipt: if `Seq <= High-Watermark`, acknowledge (PUBACK) but discard — do not process
3. If `Seq > High-Watermark`: execute business logic AND update High-Watermark atomically
4. On Epoch reset: clear the High-Watermark and perform full reconciliation

4.4 Single-Queue Multi-Consumer (SQMC)

4.4.1 Competing Consumer Mode

When a consumer subscribes with `v6-semantics: competing`:

- **Message Locking:** When a message is dispatched from the queue to a consumer at QoS 1 or 2, it is "locked" to that consumer.
- **Lock Duration:** The lock persists until the consumer sends PUBACK (QoS 1) or PUBCOMP (QoS 2).
- **Failover:** If the consumer disconnects before acknowledging, the Broker MUST immediately unlock the message and deliver it to the next available consumer in the group.
- **Round-Robin:** The Broker distributes messages across consumers in the group using round-robin or least-loaded assignment. The specific algorithm is implementation-defined.
- **Normative:** Each message MUST be delivered to exactly one consumer in a Competing Consumer group.

4.4.2 Exclusive Consumer Mode

When a consumer subscribes with `v6-semantics: exclusive`:

- The first Client to subscribe becomes the **Primary Consumer**.
- All subsequent subscribers become **Hot Standbys** — they receive no messages while the Primary is connected.
- **Normative:** If the Primary Consumer disconnects, the Broker MUST immediately promote the first Standby to Primary and resume delivery from the last acknowledged sequence.
- **Ordering guarantee:** Because only one consumer is ever active, strict ordering is preserved across failover.

---

5. Summary of Changes: v5.0 vs. v6.0

Feature	MQTT v5.0	MQTT v6.0
Protocol Level	5	**6** (or 5 with compat negotiation)
Sequencing	16-bit Packet ID (per-session, recycled)	**32-bit Stream Sequence (per-queue, persistent)**
Flow Control	Reactive push (Receive Maximum)	**Deterministic pull (FETCH / Virtual FETCH)**
Queue Type	Session-bound implicit queue	**Named durable queue (`$queue/` namespace)**
Consumer Patterns	Shared Subscriptions (loose)	**SQMC: Competing + Exclusive with message locking**
Cluster Consistency	Implementation-defined, opaque to clients	**Epoch-based failover; client-visible resync**
Ordering Guarantee	Hop-by-hop, best effort	**Global monotonic ordering within queue**
Backpressure	None (push floods consumers)	**Broker buffers; consumer controls release rate**
Rate Limiting	None	**Throughput Limit property in CONNACK**

Feature	MQTT v5.0	MQTT v6.0
New Packet Types	None (Types 1-15)	**FETCH (Type 16)**

---

6. Compatibility Layer

6.1 Negotiation

Mode	Protocol Level	User Property
Native v6.0	`6`	`mqtt-ext: v6.0` (optional, implied by level 6)
Compat v6.0 (on v5.0 broker)	`5`	`mqtt-ext: v6.0` (required)
Legacy v5.0	`5`	(absent)

6.2 Feature Mapping (Native → Compat)

Native v6.0	v5.0 Compatibility Mapping
FETCH (Type 16)	`PUBLISH` to `$SYS/queues/{name}/fetch`
Batch Size (0x42)	`User Property: ("v6-batch", "{N}")`
Stream Sequence (0x30)	`User Property: ("v6-seq", "{N}")`
Stream Epoch (0x35)	`User Property: ("v6-epoch", "{N}")`
SQMC Mode Bits	`User Property: ("v6-semantics", "competing|exclusive")`
Wait Timeout (0x36)	`User Property: ("v6-wait-ms", "{N}")`

6.3 Broker Requirements for Compat Mode

A v5.0 broker with a v6.0 Extension Plugin MUST:

1. Intercept PUBLISH packets to `$SYS/queues/*/fetch` and trigger queue message release
2. Intercept SUBSCRIBE packets containing `v6-semantics` User Properties and apply SQMC logic
3. Inject `v6-seq` and `v6-epoch` User Properties into outgoing PUBLISH packets for `$queue/` topics
4. Maintain persistent storage for `$queue/` topics independent of session state

---

7. Security

All MQTT v5.0 security requirements (TLS, Enhanced Authentication, SASL/SCRAM) are retained and remain mandatory.

[NEW for v6.0]:

- **Normative:** Access to the `$queue/` namespace MUST be controlled by ACLs. Unauthorized publish or subscribe to `$queue/` topics MUST be rejected with Reason Code `0x87 (Not Authorized)`.
- **Normative:** Access to `$SYS/queues/*/fetch` control topics MUST be restricted to authenticated clients that have completed the v6.0 handshake.
- Epoch Reset messages carry the potential to discard a client's idempotency state. Brokers MUST only send Epoch Reset signals when cluster state genuinely cannot guarantee sequence continuity. Malicious or misconfigured Epoch Resets constitute a denial-of-service vector.

Annex A: SECS/GEM Mapping

For semiconductor manufacturing environments using SECS/GEM (SEMI Equipment Communications Standard):

SECS/GEM Concept	MQTT v6.0 Mapping
Transaction ID (TID)	Stream Sequence Number (0x30)
S1F13/S1F14 (Establish Comms)	CONNECT/CONNACK with v6.0 negotiation + Epoch check
S2F21 (Remote Start)	PUBLISH to `$queue/secsgem/{device_id}/S2F21` with `v6-semantics: exclusive`
S6Fx (Event Reports / FDC)	PUBLISH to `$queue/secsgem/{device_id}/events` + Virtual FETCH for backpressure
Multi-block messages	Segmented PUBLISH with sequence continuation; reassembled via Batch Size
Link Test (Heartbeat)	FETCH with `Wait Timeout` as a “proof of life” pull
Equipment Model Reset	Epoch Reset → Client performs S1F13 re-establishment

A.1 Critical Scenario: Process Start Command (S2F21)

In SECS/GEM, S2F21 (Remote Start) is mission-critical. Loss of this command can ruin a wafer.

Without v6.0: If broker Node A crashes after accepting S2F21 but before delivery, the command may be silently lost.

With v6.0: 1. Host publishes S2F21 with `Sequence: 5001, Epoch: 1` to `$queue/secsgem/tool_001/S2F21` 2. Broker Node A persists to non-volatile storage and assigns Seq 5001 3. Node A crashes; its replica is gone but the Distributed Sequence Counter and message data survive in other nodes’ data grid replicas 4. Tool reconnects with `v6-last-seq: 5000` in CONNECT 5. Node B checks persistent queue: Seq 5001 exists and was never acknowledged 6. Node B resumes delivery: tool receives S2F21 exactly once

---

Annex B: New Reason Codes

Code	Name	Description
`0xA0`	Epoch Reset	Cluster failover caused sequence discontinuity; client must reset idempotency state
`0x9B`	Quota Exceeded	Client exceeded Throughput Limit
`0x8D`	Queue Full	Queue storage limit reached; message rejected

---

Alternative Approaches to MQTT v6.0

This document explores the design alternatives considered during the development of MQTT v6.0, explains the trade-offs, and justifies the chosen approach.

---

1. The Core Design Choice: Breaking vs. Non-Breaking

The central tension in extending MQTT is between **expressiveness** and **compatibility**. Every proposed feature was evaluated on a spectrum:

```
[Pure v5.0 Extension] ←──────────────────────→ [New Protocol Version]
   (User Properties)      (New Properties)      (New Packet Types)
      No breaks               Safe                  Breaking
```

---

2. Alternative A: Pure MQTT v5.0 Extension (Property-Based Shim)

Description

All v6.0 features encoded entirely as `User Properties` within existing v5.0 packets. No new property IDs, no new packet types.

Implementation

- Sequence numbers as `User Property: ("v6-seq", "12345")`
- Epoch as `User Property: ("v6-epoch", "2")`
- FETCH as `PUBLISH` to `$SYS/queues/{name}/fetch` with `User Property: ("v6-batch", "100")`
- Consumer modes as `User Property: ("v6-semantics", "competing")`

Pros

- **Zero compatibility risk.** Works on any MQTT v5.0 broker, any client library, today.
- **No broker changes required.** Logic lives entirely in a HiveMQ Extension Plugin.
- **Fastest time to production.** Can be deployed as a library/plugin without protocol changes.

Cons

- **Not standardizable.** User properties are opaque strings — no binary parser optimization is possible. This is a permanent workaround, not a standard.
- **Parsing overhead.** Sequence numbers as strings ("4294967295") cost 10 bytes; as 4-byte integers they cost 4 bytes + 1 ID byte = 5 bytes.
- **No protocol-level enforcement.** A broker cannot distinguish a v6.0 sequence-aware publish from a regular publish at the binary level without inspecting all user properties.
- **No `FETCH` semantics at the broker level.** The broker cannot natively throttle push delivery; it requires application-level hacks.

Verdict

Viable as a transitional shim. Adopted as the Compatibility Layer (Track B). Not suitable as the long-term specification.

---

3. Alternative B: MQTT v5.0 with Reserved Property IDs (0x30–0x45)

Description

Use the unused/reserved property ID space (`0x30` – `0x45`) to define new typed properties. No new packet types. Protocol Level remains 5.

Implementation

- `0x30` : Stream Sequence Number (Four Byte Integer)
- `0x35` : Stream Epoch (Two Byte Integer)
- `0x41` : Throughput Limit (Four Byte Integer)
- `0x42` : Batch Size (Four Byte Integer)
- FETCH still implemented as Virtual FETCH (control topic publish)

Pros

- **Binary-efficient.** Fixed-width integers instead of string user properties.
- **Fully backward-compatible.** MQTT v5.0 mandates that unknown Property IDs MUST be silently ignored by non-v6 clients and brokers.
- **Minimal parser changes.** Any existing MQTT parser that follows the spec will skip unrecognized properties safely.
- **Works without Protocol Level change.** Mixed v5.0/v6.0 deployments are seamless.

Cons

- **No native FETCH.** Without a new packet type, pull semantics must still be simulated through control topics, adding round-trip overhead.
- **Subscription option bits still blocked.** MQTT v5.0 mandates bits 6-7 of the SUBSCRIBE options byte MUST be zero; exclusive/competing modes cannot be encoded natively in the packet — only in user properties.
- **Protocol-level ambiguity.** A “Protocol Level 5 client with v6 properties” is technically a v5.0 client; there is no way to formally signal v6.0 conformance without a new level.

Verdict

Adopted as the Compatibility Extension profile. This is what Track B uses. The reserved property IDs are the right mechanism; the missing piece is the FETCH packet.

---

4. Alternative C: Native v6.0 with New Packet Type (Chosen Approach)

Description

Full protocol version bump to Level 6. Introduces FETCH as Control Packet Type 16 — previously reserved in v5.0. New subscription option bits for SQMC modes.

Implementation

- Protocol Level `6` in CONNECT
- `FETCH` packet (Type 16) as a first-class pull mechanism
- Properties `0x30`, `0x35`, `0x41`, `0x42` as typed binary fields
- SUBSCRIBE options bits 6-7 used for Competing/Exclusive flags
- `$queue/` namespace with mandatory persistence semantics

Pros

- **Full expressiveness.** Every v6.0 feature has a dedicated, binary-efficient representation.
- **Broker-level enforcement.** A v6.0 broker can natively identify `$queue/` publishes, FETCH requests, and SQMC subscriptions at the parser level without inspecting application data.
- **Formal standardizability.** A clean Protocol Level 6 specification can be submitted to OASIS as a distinct standard.
- **Performance.** FETCH as a dedicated packet eliminates the round-trip overhead of the “publish to a control topic” pattern.

Cons

- **Breaking change for v5.0 brokers.** A strict v5.0 broker closes the connection immediately upon receiving a Type 16 packet.
- **Breaking change for subscription bits.** A v5.0 broker rejects SUBSCRIBE packets with non-zero reserved bits.
- **Requires ecosystem update.** Client libraries (paho, gmqtt, Eclipse Mosquitto) must be updated to support Level 6.
- **Deployment risk.** Mixed v5.0/v6.0 environments require the Compatibility Layer.

Verdict

Chosen as the canonical v6.0 specification. The breaking changes are manageable with the Compatibility Layer (Alternative B) and negotiation handshake. The long-term correctness of a formal protocol is worth the transition cost.

---

5. Alternative D: Replace MQTT Entirely (Kafka / AMQP 1.0 / NATS JetStream)

Description

Instead of extending MQTT, adopt a different messaging protocol that already provides the required industrial features.

Comparison

Feature	MQTT v6.0	Apache Kafka	AMQP 1.0	NATS JetStream
Lightweight clients	Yes	No	Partial	Yes
Edge/IoT footprint	Excellent	Poor	Moderate	Good
Named durable queues	Yes (v6.0)	Yes (Topics)	Yes	Yes
Pull-based consumption	Yes (FETCH)	Yes (Poll API)	Yes	Yes
32-bit+ sequencing	Yes (v6.0)	Yes (Offset)	No native	Yes
Cluster consistency	Epoch-based	ISR/Leader	Partition leader	Raft consensus
Existing MQTT ecosystem	Full compatibility	None	Partial bridge	None
SECS/GEM mapping	Native	Requires adapter	Requires adapter	Requires adapter
Standard body	OASIS	Apache/Confluent	AMQP WG	CNCF

Analysis

Kafka provides everything v6.0 proposes, but: - Kafka clients are heavyweight (JVM, 50MB+). Not viable for edge devices or microcontrollers. - The existing investment in MQTT infrastructure (broker, client libraries, security tooling) is significant. - SECS/GEM integration with non-MQTT protocols requires a translation adapter,

introducing latency and another failure point. - NATS JetStream is compelling for cloud-native workloads but lacks the industrial IoT ecosystem MQTT has built.

Verdict

Not adopted. The v6.0 extensions preserve the edge/IoT deployment model of MQTT while adding the industrial reliability features. Replacing MQTT would abandon its core advantage in constrained environments.

---

6. Alternative E: Extend MQTT Shared Subscriptions (v5.0 `$share/`)

Description

Enhance the existing MQTT v5.0 `$share/group/topic` mechanism to add message locking, exclusive consumer semantics, and failover logic — without changing the core protocol.

Pros

- Uses an existing, widely-supported mechanism
- No new packet types or property IDs required
- Implementable today as a broker plugin

Cons

- `$share/` groups are still session-dependent — the queue vanishes when the last client disconnects
- No way to add explicit sequencing or gap detection within the `$share/` spec
- No standardized way to express “exclusive” (hot-standby) mode in the subscription syntax
- Extending `$share/` syntax creates a non-standard variation that other broker implementations won’t support

Verdict

Partially adopted. The conceptual model of consumer groups and competing delivery from `$share/` inspired the SQMC design. But v6.0 requires named queue persistence and sequencing that `$share/` fundamentally cannot provide without a new namespace.

---

7. Design Decision Matrix

Criterion	Pure User Props	Reserved Props (0x30+)	Native v6.0	Kafka/NATS	$share/ Ext
v5.0 Wire Compat	✅ Full	✅ Full	⚠️ Breaking	❌ None	✅ Full
Binary Efficiency	❌ Poor	✅ Good	✅ Best	✅ Best	✅ Good
Native FETCH	❌ No	❌ No	✅ Yes	✅ Yes	❌ No
Standardizable	❌ No	⚠️ Partial	✅ Yes	✅ Yes	❌ No
Edge/IoT Viable	✅ Yes	✅ Yes	✅ Yes	❌ No	✅ Yes
SQMC Enforcement	❌ App-layer	⚠️ Partial	✅ Protocol	✅ Protocol	⚠️ Partial

Criterion	Pure User Props	Reserved Props (0x30+)	Native v6.0	Kafka/NATS	$share/ Ext
Queue Persistence	❌ Plugin only	⚠️ Plugin only	✅ Spec-mandated	✅ Spec-mandated	❌ No
Time to Deploy	✅ Immediate	✅ Immediate	⚠️ Ecosystem update	❌ Full migration	✅ Immediate

Recommendation: Native v6.0 for new deployments with a full MQTT v6.0 ecosystem. Compatibility Layer (Reserved Props) for transitional deployments. Pure User Props shim for proof-of-concept and testing only.

Critical Perspectives, Rebuttals & Technical Pitfalls

This document provides an honest assessment of the MQTT v6.0 proposal from multiple perspectives: MQTT standards committee members, IoT architects, software engineers, and SREs. For each criticism, a rebuttal and/or mitigation is provided.

Part 1: Standards Committee & Architecture Critique

Criticism 1: "MQTT's success is its simplicity. v6.0 turns it into a bloated Kafka clone."

From: MQTT protocol purists, standards committee members

The Argument: MQTT succeeded precisely because it fits in 128KB of RAM and works on a microcontroller. By adding 32-bit sequencing, Epochs, persistent queues, and FETCH semantics, we are violating the core design principle. The protocol will lose its competitive advantage against AMQP and become yet another "enterprise messaging platform."

Rebuttal:

1. **"Context Isolation" Defense:** These features are strictly opt-in. A temperature sensor publishing to `temperature/office` uses MQTT v6.0 identically to MQTT v3.1.1 — no sequence numbers, no epochs, no FETCH. The protocol grows upward, not outward. A compliant v6.0 client that doesn't use `$queue/` is binary-indistinguishable from a v5.0 client.

2. **"Standardization of Hacks" Argument:** Every major industrial MQTT deployment has already built these features — in the payload. Sequence IDs, idempotency keys, and consumer group logic are already in production payloads at industrial scale. v6.0 moves these de-facto requirements into the protocol header where brokers can optimize for them. The complexity exists; v6.0 just standardizes it.

3. **Scope Comparison:** The jump from v5.0 to v6.0 is smaller than the jump from v3.1.1 to v5.0. The v5.0 spec added Properties, Reason Codes, Shared Subscriptions, User Properties, and Will Delay — all of which "bloated" the protocol by the same argument. The ecosystem absorbed it; industrial users demanded it.

Criticism 2: "The `$queue/` namespace breaks topic hierarchy semantics."

From: MQTT architects, broker implementers

The Argument: MQTT topics are flat strings with hierarchical separators. The `$` prefix convention is fragile — `$SYS/` is implementation-specific, `$share/` is standard but limited. Creating a new `$queue/` namespace adds confusion, may conflict with existing broker topic trees, and is a "soft break" for v5.0 brokers that might route or reject it incorrectly.

Rebuttal:

1. **Existing precedent.** The `$SYS/` and `$share/` namespaces already demonstrate that `$`-prefixed topics are an accepted mechanism for protocol-level semantics. `$queue/` follows the same pattern.
2. **Explicit negotiation mitigates misrouting.** The v6.0 handshake (Section 3.1 of the spec) requires clients to confirm broker v6.0 support before using `$queue/`. If the broker does not confirm, the client MUST NOT use `$queue/` — preventing silent misrouting.
3. **ACL isolation.** The spec mandates `$queue/` access be controlled via ACLs, just like `$SYS/`. A v5.0 broker that doesn't understand `$queue/` will simply apply its default ACL policy (typically: reject `$`-prefixed topics from unprivileged clients), preventing message loss by failing loudly.

Remaining Risk: A misconfigured v5.0 broker with open ACLs COULD treat `$queue/foo` as a literal topic and accept publishes, creating a silent reliability hole. **Mitigation:** Document that `$queue/` topics MUST be ACL-blocked on v5.0 brokers.

Criticism 3: "The FETCH packet (Type 16) is a hard break that undermines the whole 'compatible' story."

From: Standards committee, library maintainers

The Argument: Type 16 was reserved in v5.0 specifically to indicate an error. A v5.0 broker receiving a Type 16 packet MUST close the connection. In a heterogeneous environment with mixed v5.0 and v6.0 brokers, a single misrouted FETCH packet causes cascading disconnections.

Rebuttal:

1. **The Compatibility Layer exists for this reason.** The Virtual FETCH mechanism (Section 4.2.2) provides identical semantics using standard v5.0 PUBLISH packets. No environment is forced to use native FETCH.
2. **Protocol-level negotiation prevents misrouting.** A client sets Protocol Level 6 in CONNECT. A v5.0 broker will reject the connection at the CONNECT stage with `Reason Code 0x84 (Unsupported Protocol Version)` — before any FETCH packet is ever sent. The client then falls back to Compatibility Mode. There is no scenario where a FETCH packet reaches a v5.0 broker in a correctly implemented client.
3. **Clean version separation.** Having a Level 6 that is truly distinct from Level 5 is better long-term than a "v5.1" that silently extends in ways that break some brokers but not others. The break is explicit and managed.

Part 2: Software Engineering & Performance Critique

Criticism 4: "The original proposal's Leader Node model contradicts HiveMQ's masterless architecture."

From: HiveMQ engineers, distributed systems architects

The Argument: The early draft proposed assigning a "Leader Node" per `$queue/` via consistent hashing, with all publishes routing through that node. HiveMQ is explicitly masterless — all nodes are peers, no consensus protocol (Raft/Paxos) is used, and no node "owns" a topic. The Leader Node model would require HiveMQ to fundamentally change its architecture.

Resolution (accepted — the proposal was updated):

This criticism is **correct**. The Leader Node model was removed from the specification in favour of a **Distributed Sequence Counter** approach that is native to HiveMQ's actual architecture:

1. **Distributed Sequence Counter.** Each `$queue/` maintains a single cluster-wide atomic counter in HiveMQ's replication data grid. Any node can increment it atomically (compare-and-swap) to claim the next sequence number. No routing changes are required; any node handles any publish. HiveMQ's existing data grid already provides the distributed primitives needed.

2. **No leader election, no Raft/Paxos.** The atomic counter replaces the entire leader/follower model. Split-brain sequencing (two nodes issuing the same Seq) is prevented by the CAS semantics of the counter, not by funneling all traffic through one node.

3. **Epoch triggers change.** Instead of "leader failover triggers Epoch++", the trigger is "replication quorum loss" — i.e., when a network partition resolves and the counter's replicated state cannot be verified. This maps directly to HiveMQ's partition detection events, which the Extension SDK already exposes.

4. **Hot-queue scaling still works.** If a single `$queue/` becomes a throughput bottleneck, the correct mitigation is application-level sharding (`$queue/line_1_A` , `$queue/line_1_B`) — exactly as you would shard a Kafka topic into multiple partitions. The counter CAS overhead per message is negligible compared to the NVMe write latency.

Criticism 5: "Replication lag will cause data loss in the replication gap window."

From: SREs, database engineers

The Argument: Node A received a publish and incremented the sequence counter to Seq 100, but crashes before the message is replicated to other nodes. Node B has no record of Seq 100. A client that was tracking Seq 100 will detect a gap, but the message is unrecoverable.

Rebuttal:

1. **Quorum write prevents the gap.** The spec requires the PUBACK to the publisher only after the message is confirmed on `N` replicas (configurable `Replication-Factor`). With `Replication-Factor: 2` , Node A crashing never loses the message because Node B already has it before the publisher received its PUBACK.

2. **The atomic counter and quorum write are decoupled.** The sequence number is claimed first (atomic CAS on the counter), then the message is written to quorum. If Node A crashes between those two steps, the claimed Seq number is "orphaned." This creates a one-time gap. The Epoch does NOT need to increment in this case — the surviving cluster knows the gap exists and can communicate it to consumers when they request that sequence range.

3. **Epoch for partition events only.** Epoch increments only when the entire counter quorum is lost (partition event) — not for individual node crashes. This reduces spurious full-resyncs that would otherwise be triggered by routine node restarts.

4. **The v5.0 baseline is worse.** A v5.0 broker silently loses the message with no signal to the client. v6.0 makes the gap explicit, auditable, and recoverable at the application layer.

Criticism 6: "Persistent queues on the broker will cause storage bloat and operational nightmares."

From: SREs, operations teams

The Argument: If every `$queue/` is persisted to non-volatile storage forever, brokers will eventually run out of disk space. "Leaked queues" (queues no one is reading) will accumulate indefinitely. This is an operational disaster in a long-running production system.

Rebuttal:

1. **Mandatory TTL and max-size properties.** The spec requires `v6-queue-ttl` (how long to hold the queue after last consumer disconnect) and `v6-queue-max-size` (message count limit). Both default to implementation-defined values but MUST be configurable. The spec also requires a "drop-oldest" or "reject-new" policy when limits are reached — no silent unbounded growth.

2. **"Tiered Storage" implementation model.** Production brokers (analogous to HiveMQ or Kafka) should implement hot/cold storage: recent messages in RAM (or NVMe), deep backlogs tiered to object storage (S3/GCS). The broker is a transient buffer with bounded local cost.

3. **Operational observability.** 32-bit sequences make "leaked queue" detection trivial: an SRE can query `MAX(Seq) - Last_Consumer_ACK_Seq` to calculate exact backlog depth per queue in real-time. This is far better than the current situation where session-bound queues silently accumulate with no tooling to measure them.

Part 3: SRE & Operational Critique

Criticism 7: "The FETCH mechanism adds latency compared to push delivery."

From: Performance engineers, real-time systems architects

The Argument: A FETCH round-trip adds 2× the one-way latency compared to a push delivery. For real-time applications, this is unacceptable.

Rebuttal:

1. **Scope clarification.** FETCH is exclusively for `$queue/` topics — durable industrial queues. Standard pub/sub topics (real-time sensor telemetry, command delivery to online devices) continue to use push delivery unchanged. The proposal does not change push semantics for non-queue topics.

2. **Cost-benefit for industrial queues.** In semiconductor FDC or financial messaging, "missing a message" is infinitely more expensive than 5ms of added latency. The FETCH mechanism prevents the host system from crashing under data spikes — which is the #1 cause of unplanned downtime in industrial MQTT deployments.

3. **Long-polling mitigates latency for near-real-time queues.** The `Wait Timeout` property in the FETCH packet enables long-polling: the broker holds the connection open and responds immediately when a new message arrives. Effective latency is near-zero for well-paced queues.

Criticism 8: "Custom property IDs in the reserved range could conflict with future OASIS standardization."

From: Standards committee members, protocol engineers

The Argument: Properties `0x30` – `0x45` are currently unassigned in MQTT v5.0, but they could be assigned by a future OASIS revision. If v6.0 squats on these IDs and a future v5.1/v6.x assigns different semantics to the same IDs, we have a compatibility disaster.

Rebuttal:

1. **This is exactly why we need formal standardization.** The solution to "these IDs might be assigned later" is to submit v6.0 to OASIS and have the IDs formally reserved. The alternative — using User Properties forever — permanently forecloses binary optimization.

2. **The proposal is already in "draft OASIS" format.** The spec is structured to be submitted as an OASIS draft. Early submission reserves the property IDs before any conflict can arise.

3. **Interim risk is manageable.** During the pre-standardization period, the Compatibility Layer (User Properties) can be used exclusively. The reserved property IDs are only activated in native v6.0 brokers that understand the spec. A v5.0 broker will silently ignore them, not misinterpret them.

Technical Pitfalls

Pitfall 1: Sequence Wraparound at 2^32

Description: A 32-bit sequence number wraps around after 4,294,967,295 messages. A high-throughput queue generating 100,000 messages/second exhausts the sequence space in ~43,000 seconds (~12 hours).

Mitigation: - For high-throughput queues, the broker MUST handle wraparound gracefully by treating `Seq 0` after `Seq 0xFFFFFFFF` as a continuation (not a gap). - For disambiguation, wraparound can be coupled with an Epoch increment — a full 32-bit cycle increments the Epoch. This gives an effective 48-bit sequence space. - A future revision may introduce 64-bit sequences for ultra-high-throughput use cases (see Annex B of the spec).

Pitfall 2: Idempotency Table Growth

Description: Consumers maintaining a “processed messages” idempotency table (for exactly-once semantics) may grow the table unboundedly if sequence numbers are not garbage-collected.

Mitigation: - The High-Watermark model (store only the latest processed Seq) is sufficient for in-order delivery queues — O(1) storage. - Only out-of-order delivery (not expected from ordered `$queue/` topics) requires a full set-based idempotency table. - Epoch Resets provide natural garbage-collection boundaries: on Epoch change, discard the entire idempotency table.

Pitfall 3: Competing Consumer Fairness

Description: Round-robin distribution may create consumer starvation if one consumer processes messages slowly. Slow consumers receive fewer messages but still hold “locks” on in-flight messages, blocking queue throughput.

Mitigation: - The spec allows brokers to implement “least-loaded” instead of strict round-robin: distribute the next message to the consumer with the fewest unacknowledged messages. - A per-consumer `Receive Maximum` limit (inherited from v5.0) caps the number of in-flight locks per consumer. - Consumers that are consistently slow should be flagged via broker metrics and operator intervention.

Pitfall 4: Epoch Reset as a DoS Vector

Description: A compromised or misconfigured broker could send spurious Epoch Reset signals, causing all connected clients to discard their idempotency state and re-synchronize simultaneously — a thundering herd.

Mitigation: - The spec requires Epoch increments only when the broker cannot guarantee sequence continuity (Section 4.3.2). Any other Epoch increment is a protocol violation. - Clients SHOULD implement Epoch change rate limiting: more than N Epoch changes per hour is anomalous and should trigger an alert and/or connection refusal. - Epoch changes MUST be logged with timestamps and node identifiers for forensic auditing.

Pitfall 5: Virtual FETCH Overhead (Compat Mode)

Description: The Virtual FETCH mechanism publishes a message to `$SYS/queues/{name}/fetch` and waits for the broker to respond with the batch. This is two round-trips per batch plus the overhead of the control topic publish.

Mitigation: - Use the `Wait Timeout` user property with long-polling to reduce idle round-trips. - For high-throughput applications, pipeline FETCH requests (send the next FETCH before the current batch is fully acknowledged). - This overhead is the explicit trade-off of Compatibility Mode. Native v6.0 FETCH eliminates it; operators should plan to migrate to Protocol Level 6 for production high-throughput deployments.

Pitfall 6: `$queue/` Namespace Collision with Existing Topics

Description: Deployments that already use topic paths containing the string `queue` (e.g., `myapp/queue/events`) may have topic tree management issues or operator confusion.

Mitigation: - The `$queue/` prefix begins with `$`, which is already a reserved character in MQTT (standard topics never start with `$`). No collision with user-defined topic spaces is possible. - Operator documentation should clearly distinguish `$queue/` (protocol-level named queues) from `myapp/queue/` (application-level topic paths).

Summary: Risk Matrix

Criticism / Pitfall	Severity	Probability	Mitigation Quality
Type 16 breaks v5.0 brokers	High	High (mixed env)	✅ Full (compat layer)
Leader node model (removed)	High	Resolved	✅ Full (distributed counter)
Replication lag / data loss	High	Low (with quorum)	✅ Good (epoch + quorum)
Storage bloat	Medium	Medium	✅ Good (TTL + limits)
Sequence wraparound (32-bit)	Low	Low (normal scale)	✅ Good (epoch cycle)
Epoch Reset DoS	Medium	Low	⚠️ Partial (rate limiting)
SUBSCRIBE bits break v5.0	High	High (native mode)	✅ Full (user props)
Property ID conflict (OASIS)	Medium	Low	✅ Good (formalize early)
Idempotency table growth	Low	Low (ordered queue)	✅ Good (high-watermark)
Virtual FETCH overhead	Low	High (compat mode)	⚠️ Partial (pipeline)

MQTT v6.0 Performance Analysis: High-Volume Use Cases

> ***Focus:*** *Large distributed energy providers (DEPs), grid SCADA, and industrial IoT at scale — characterised by millions of sensors, sub-second telemetry, burst fault events, and mandatory regulatory audit trails.*

---

1. Workload Profile

Dimension	Typical DEP Scale
Sensors / endpoints	1M–10M (smart meters, PMUs, RTUs, substations)
Steady-state throughput	500k–3M msg/sec cluster-wide
Burst (fault/storm event)	10×–50× steady state for 30–120 seconds
Message size	64–512 bytes (telemetry); 1–50KB (commands, recipes)
Latency requirement	<10ms for control commands; 100–500ms acceptable for telemetry
Retention	90 days regulatory minimum; 7 years for audit
Consumer count	10–200 competing consumers per queue (analytics pipelines)

---

2. Performance Bottlenecks in the Current Proposal

2.1 CAS Counter Contention at Scale

Mechanism: The Distributed Sequence Counter requires an atomic CAS increment on the cluster data grid for every `$queue/` PUBLISH. Under contention, CAS degrades from O(1) to O(retries × RTT) as threads spin on the atomic variable.

Impact at DEP scale: A single `$queue/scada/events` receiving 500k msg/sec from 50k RTUs means ~10k CAS operations per second per broker node competing for the same counter. At 1ms inter-node RTT, contention-driven retry cycles will cap throughput well below the theoretical maximum.

Proposed fix — Partitioned Sequence Spaces:

Assign each broker node a pre-allocated sequence range of size `W` (the "window size"). Nodes increment locally without cross-node CAS until they exhaust their window, then claim the next window atomically.

```
Node 1 window: Seq 1 – 1,000,000        (local counter, no CAS)
Node 2 window: Seq 1,000,001 – 2,000,000 (local counter, no CAS)
Node 3 window: Seq 2,000,001 – 3,000,000 (local counter, no CAS)
```

- Window claiming (CAS) is amortized across `W` messages — at W=1M and 500k msg/sec, each node claims a new window every ~2 seconds.
- Gaps between windows are valid: Seq 1,000,000 → 1,000,001 is a valid "gap" — consumers must tolerate intra-window ordering while the global order is maintained at window boundaries.
- Window size `W` should be a configurable queue property (`v6-seq-window-size`).

New spec normative: The Stream Sequence Number MUST be monotonically increasing within a node's assigned window. Global monotonicity across windows is RECOMMENDED but not REQUIRED — consumers MUST treat any Seq > HWM as processable regardless of gap size when the queue uses partitioned sequence windows.

2.2 High-Watermark Inadequacy Under Burst

Current spec: Consumer maintains a single `High-Watermark` integer (last processed Seq). Works for ordered, serial consumption.

Problem at DEP scale: During a grid fault event, 50 competing consumers receive messages in parallel. In-flight Seq values span a wide range simultaneously:

```
Consumer 1: processing Seq 10,001 (slow — database write)
Consumer 2: processing Seq 10,050 (fast)
Consumer 3: processing Seq 10,100 (fast)
```

If Consumer 2 and 3 finish before Consumer 1, their HWM would advance past Consumer 1's in-flight message. If Consumer 1 then crashes, the group HWM is ambiguous — "did we process 10,001?"

Proposed fix — Sliding Window HWM:

Replace the single HWM integer with a **min/max window** tracked per consumer group at the broker:

```
Group HWM = {
  committed:  10,000   (all Seq ≤ this are acknowledged by all group members)
  in_flight:  [10,001, 10,002, ..., 10,150]  (dispatched, awaiting PUBACK)
  max_dispatched: 10,150
}
```

- `committed` advances only when all Seq below the next gap are acknowledged.
- The broker tracks in-flight locks as a bitset bounded by `Receive Maximum`.
- Consumer crash recovery: any in-flight Seq belonging to the crashed consumer is re-dispatched to the next available consumer.
- Client-side HWM for exactly-once deduplication remains a single integer (`committed`), not the full window.

New spec normative: Brokers implementing SQMC competing consumer mode MUST track a per-group `committed_hwm` (the highest Seq for which all lower Seqs are acknowledged) in addition to the per-consumer in-flight lock set.

2.3 Quorum Write Latency vs. Real-Time Telemetry

Current spec: All `$queue/` PUBLISH operations require a quorum write before PUBACK. At replication factor 2 and 1ms inter-node RTT, this adds 2–5ms per message.

Impact at DEP scale: PMU (Phasor Measurement Unit) data arrives at 60 samples/sec per device. A 2–5ms PUBACK delay is acceptable. But for 50k PMUs generating 3M msg/sec total, quorum write serialisation creates head-of-line blocking in the broker's replication pipeline — effective throughput drops by 40–60% compared to async replication.

Proposed fix — Durability Tier Namespaces:

Split the `$queue/` namespace into three tiers with different durability contracts:

Namespace	Durability	Write Path	Use Case
`$queue/`	Synchronous quorum	PUBACK after N replicas	Commands, audit, SECS/GEM S2F21
`$stream/`	Async replication	PUBACK after local write; replication in background	High-frequency telemetry, PMU data
`$log/`	Append-only, eventual	PUBACK immediately; batched replication	FDC data, bulk sensor history

- The same FETCH, Seq, and Epoch semantics apply to all three tiers.
- `$stream/` and `$log/` topics carry a `Durability` property (`0x43`) in PUBLISH indicating the tier: `0x01` = sync quorum, `0x02` = async, `0x03` = eventual.

- Consumers subscribing to `$stream/` or `$log/` MUST tolerate occasional Seq gaps (due to async write failures) and SHOULD NOT use exactly-once semantics based on Seq alone.

2.4 FETCH Batch Size Tuning for Burst Recovery

Current spec: `Batch Size` is a fixed integer per FETCH request.

Problem: During burst recovery (after a 60-second grid storm event), a consumer with a 10M-message backlog issuing `batch=100` requests requires 100k round-trips to drain the queue. At 10ms RTT, that is 16 minutes of drain time.

Proposed fix — Adaptive Batch Size:

Add two optional FETCH properties:

- `v6-batch-max` : Upper bound on batch size the consumer will accept (consumer protection).
- `v6-batch-target-ms` : Target delivery window in milliseconds. The broker calculates the batch size needed to fill the window based on queue throughput, bounded by `v6-batch-max` .

For a consumer that wants "deliver as many as I can process in 500ms", the broker calculates:

```
batch = min(v6-batch-max, queue_throughput_per_ms × 500)
```

This eliminates the need for consumers to manually tune batch sizes as queue depth fluctuates.

2.5 Rate Limiting Granularity

Current spec: Throughput Limit (Property `0x41`) is per-connection in KB/s.

Problem for DEPs: A DEP SCADA system has thousands of client connections sharing the same broker. A per-connection limit cannot prevent a single rogue data pipeline from saturating broker NIC bandwidth while legitimate control traffic is dropped.

Proposed fixes:

1. **Per-queue rate limit:** Add a `v6-queue-rate-limit` User Property to SUBSCRIBE, specifying max msgs/sec the broker will release from that queue to this consumer group — independent of connection-level limits.

2. **Priority-based admission:** High-priority messages (`v6-priority: 255`) bypass the rate limiter. This ensures S2F21 (process start) commands are never delayed by FDC data storms.

3. **Credit-based flow control:** Replace the token-bucket CONNACK approach with an explicit credit system:

   - Broker sends `CREDIT {queue_name, msgs_per_sec}` in CONNACK or a new control topic.
   - Consumer sends `CREDIT_RETURN` when it can accept more.
   - This is analogous to AMQP 1.0's link credits and prevents both flooding and starvation.

2.6 Epoch Reset Thundering Herd

Current spec: On Epoch reset, all consumers connected to the affected queue simultaneously discard their HWMs and begin full resyncs.

Problem at DEP scale: With 200 competing consumers on `$queue/grid/events`, a single partition event triggers 200 simultaneous full resyncs — each reading the queue from Seq=0. This is a thundering herd that can exhaust broker I/O for minutes.

Proposed fix — Staggered Resync:

Add a `v6-resync-jitter-ms` property to DISCONNECT (Reason Code 0xA0). Each consumer receives a different jitter value (e.g., uniformly distributed over 0–30,000ms). Consumers delay their CONNECT retry and resync by this value. The broker staggers the resyncs, preventing I/O saturation.

For the broker to generate useful jitter values:

```
jitter_for_consumer_i = (consumer_group_size × resync_window_ms / N) × i
```

where `i` is the consumer's ordinal within the group (known from SQMC group membership).

---

3. Updated Property Table

ID	Name	Type	Packets	Purpose
`0x30`	Stream Sequence	4B Int	PUBLISH	Monotonic message ID
`0x35`	Stream Epoch	2B Int	PUBLISH, CONNACK	Cluster era
`0x41`	Throughput Limit	4B Int (KB/s)	CONNACK	Per-connection rate cap
`0x42`	Batch Size	4B Int	FETCH	Max messages per batch
`0x43`	**Durability Tier**	1B Enum	PUBLISH	`0x01` sync, `0x02` async, `0x03` eventual
`0x44`	**Batch Target Ms**	2B Int	FETCH	Target delivery window for adaptive batching
`0x45`	**Resync Jitter Ms**	2B Int	DISCONNECT (0xA0)	Staggered resync delay

---

4. Namespace Additions

Namespace	Durability	Sequence	Exactly-Once
`$queue/`	Sync quorum	Strict monotonic	Supported
`$stream/`	Async replicated	Best-effort monotonic	Not guaranteed
`$log/`	Eventual / append	Monotonic within node	Not supported
`$SYS/queues/*/fetch`	N/A (control)	N/A	N/A

---

5. Summary: Adaptations for DEP Scale

Problem	Scale Trigger	Fix
CAS counter contention	>100k msg/sec per queue	Partitioned sequence windows

Problem	Scale Trigger	Fix
		(`v6-seq-window-size`)
Single-integer HWM inadequate	>10 competing consumers	Sliding window HWM — committed + in-flight bitset
Quorum write latency	>1M msg/sec telemetry	Durability tier namespaces (`$stream/` , `$log/`)
Slow backlog drain	Post-burst recovery	Adaptive batch size (`v6-batch-target-ms`)
Per-connection rate limit too coarse	Thousands of connections	Per-queue rate limit + priority admission bypass
Epoch reset thundering herd	>50 consumers, partition event	Staggered resync with `v6-resync-jitter-ms`

MQTT v6.0 Architecture Diagrams

---

1. System Context (C4)

MQTT v6.0 — System Context

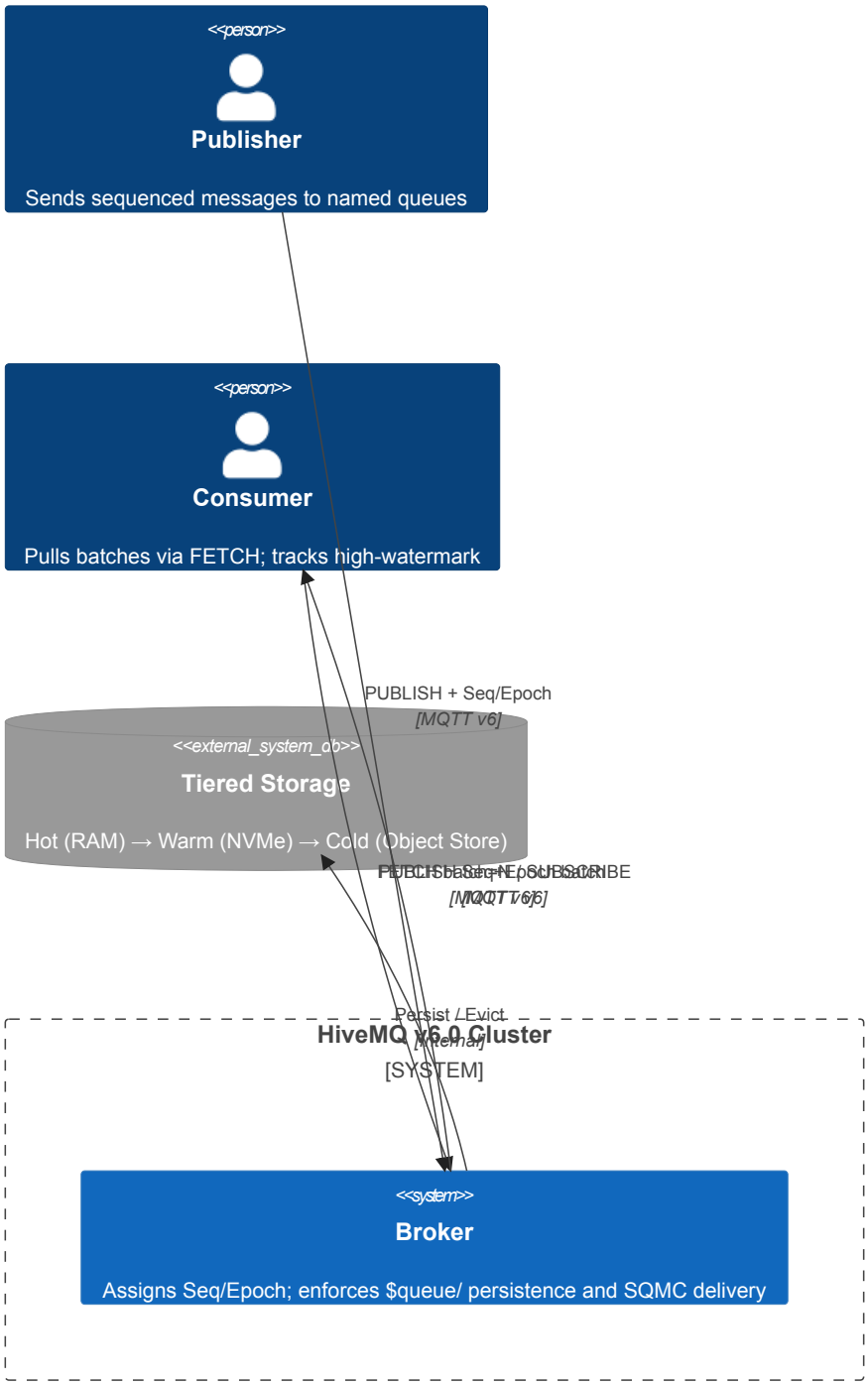

Diagram 1

2. Container View: HiveMQ v6.0 (C4)

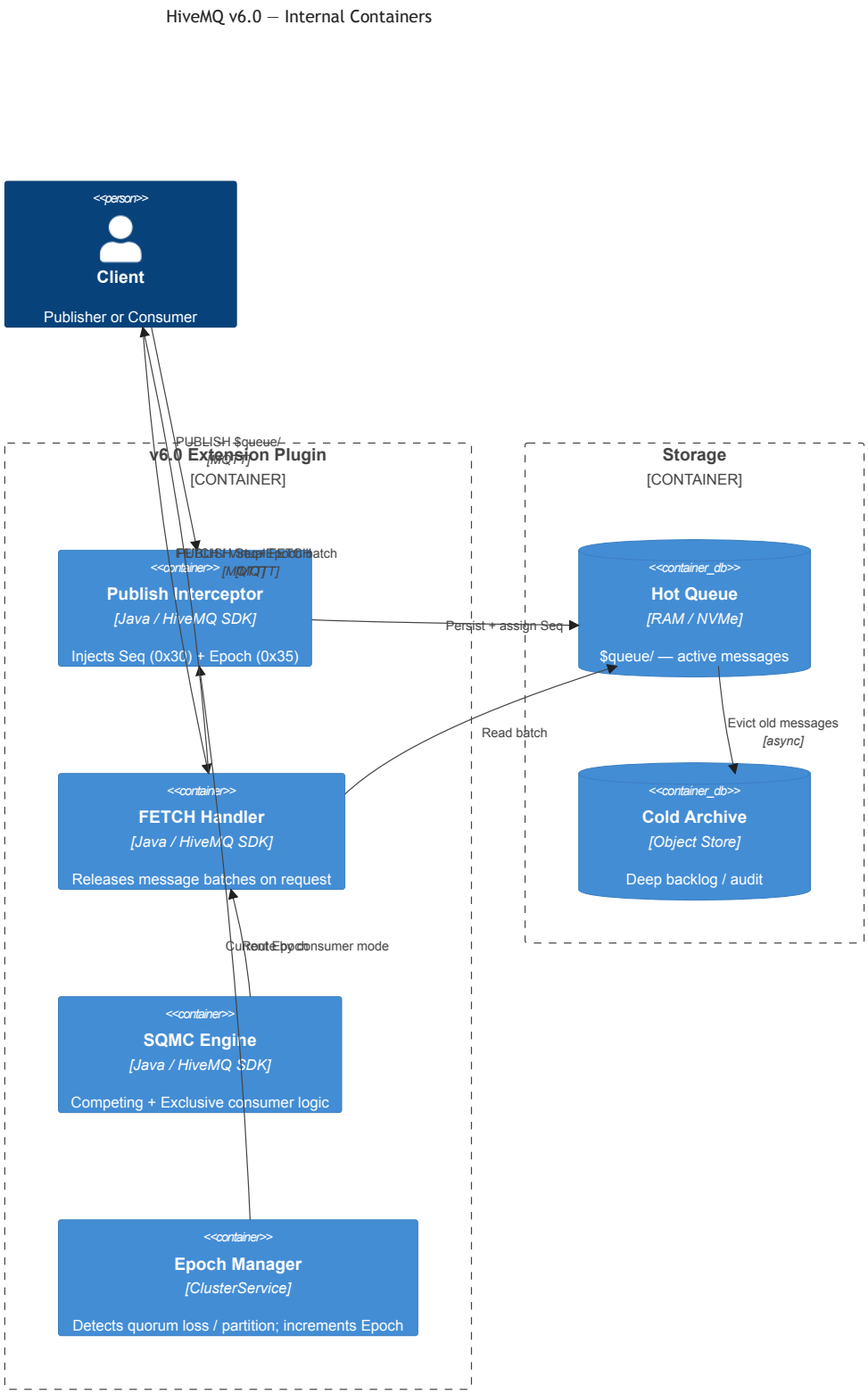

Diagram 2

3. Feature Layers

Diagram 3

4. New v6.0 Properties

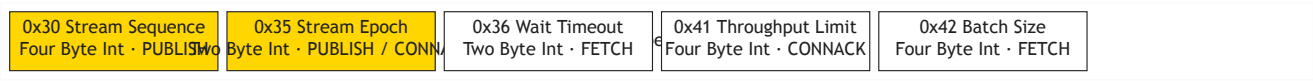

Diagram 4

MQTT v6.0 Packet Flow Diagrams

1. Connection Negotiation

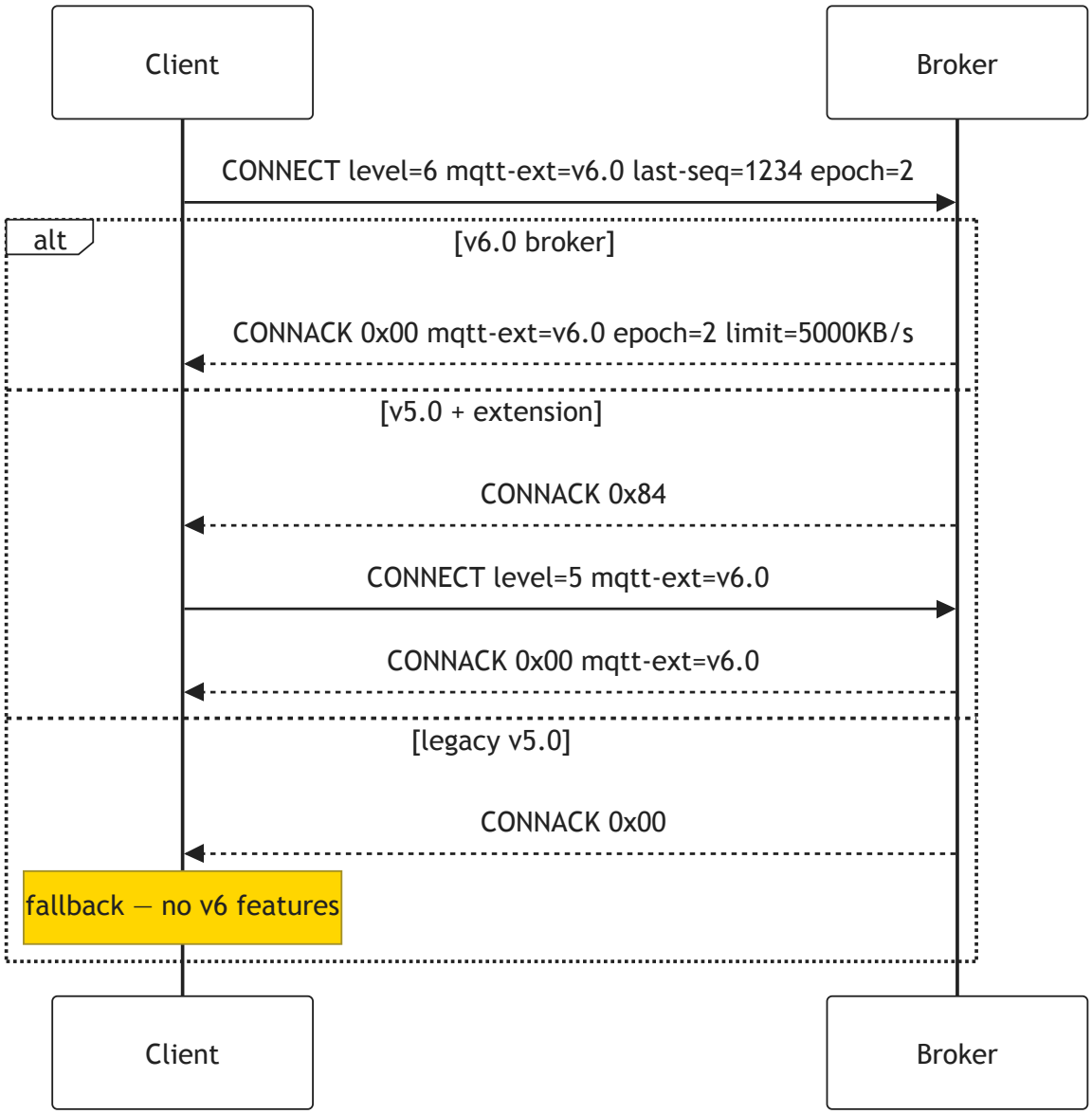

Diagram 5

2. Publish → Sequence Assignment (masterless)

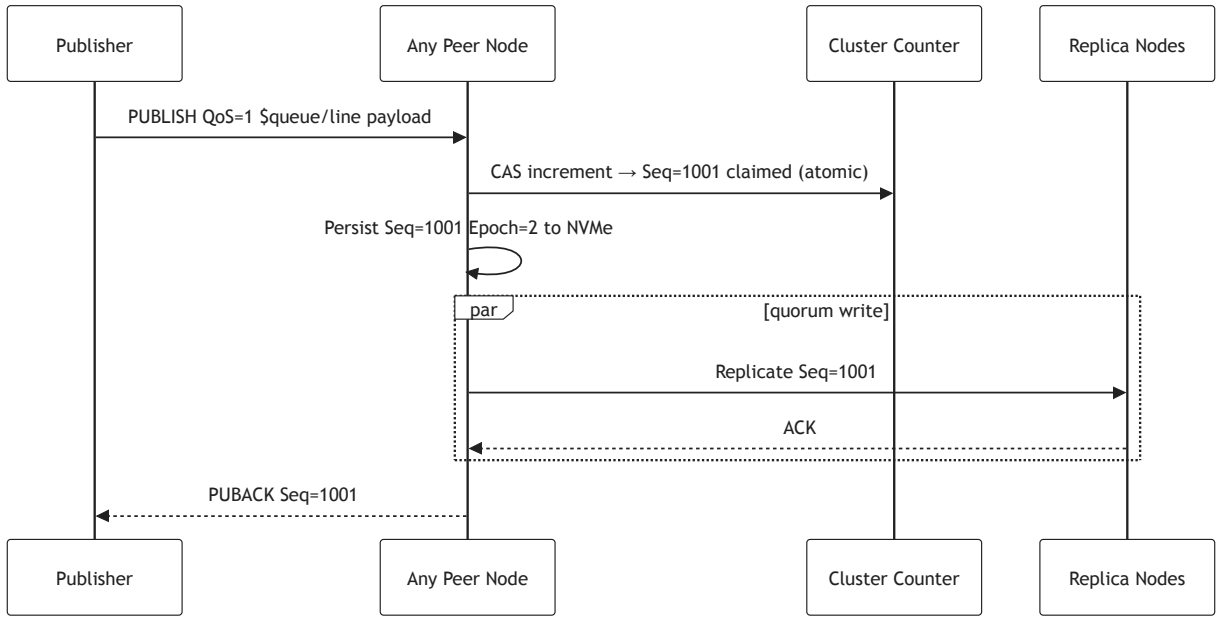

Diagram 6

3. Native FETCH (Protocol Level 6)

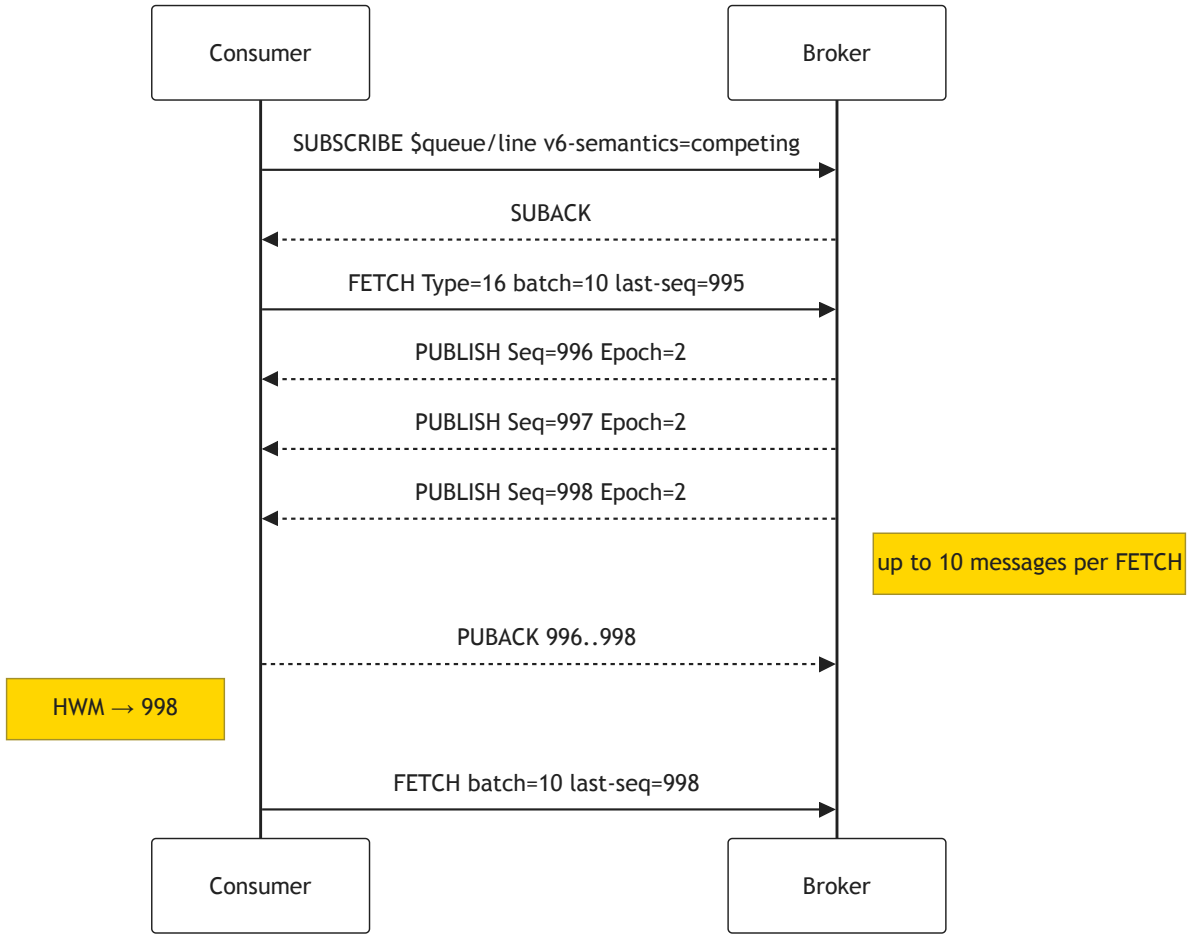

Diagram 7

4. Virtual FETCH (Compat Mode)

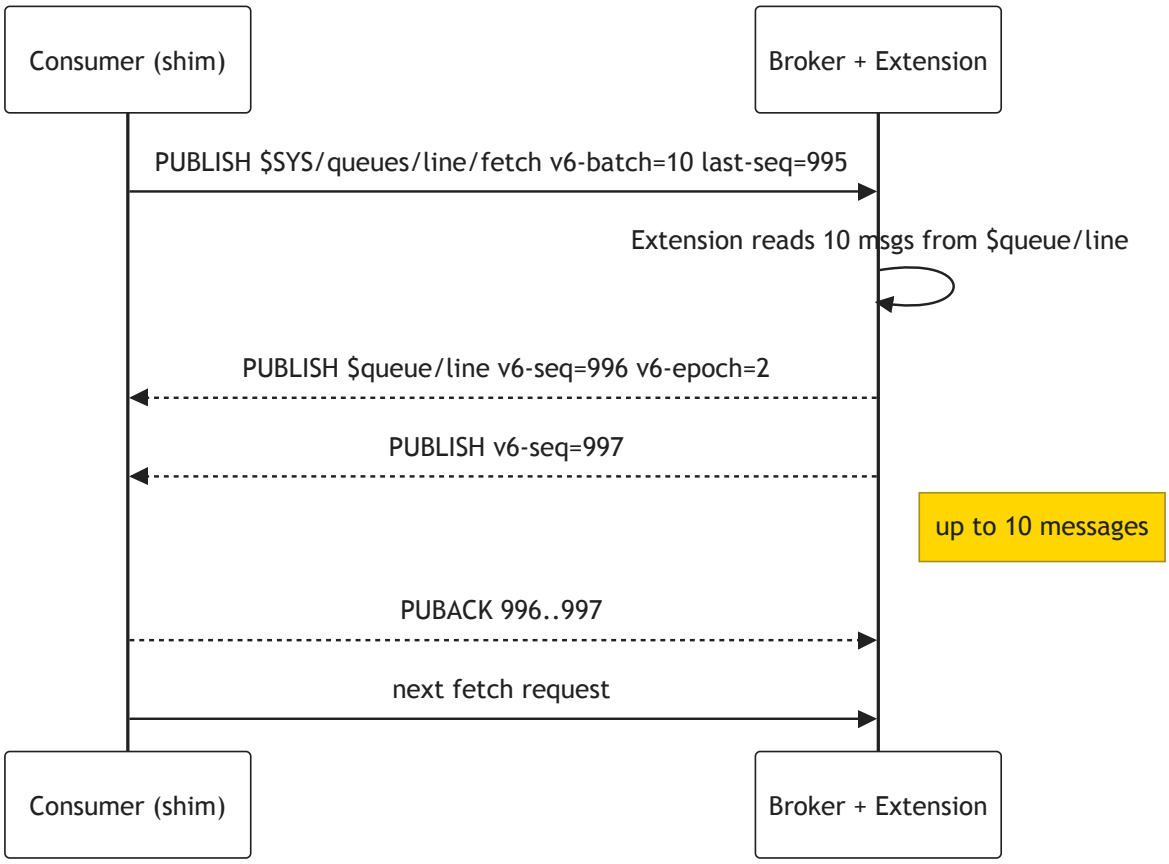

Diagram 8

5. Competing Consumer — Failover

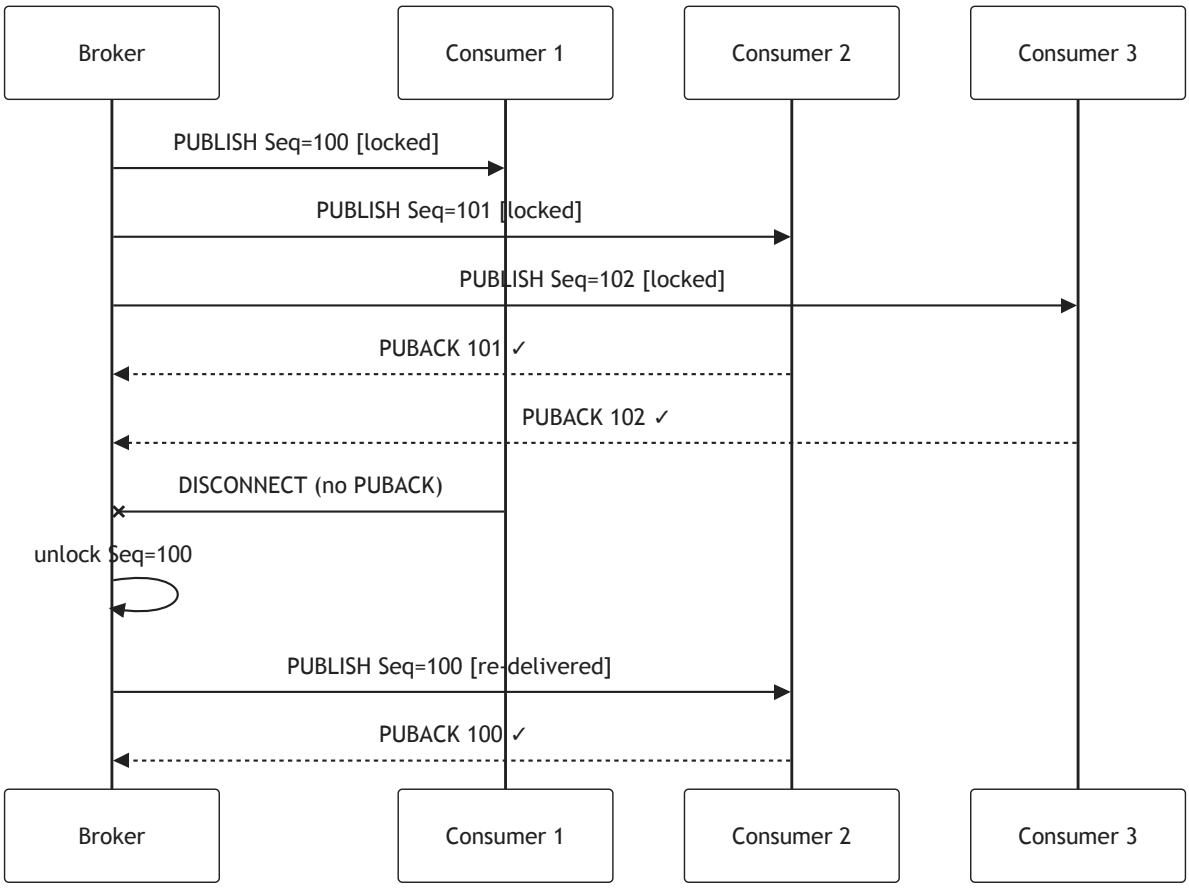

Diagram 9

6. Exclusive Consumer — Hot-Standby

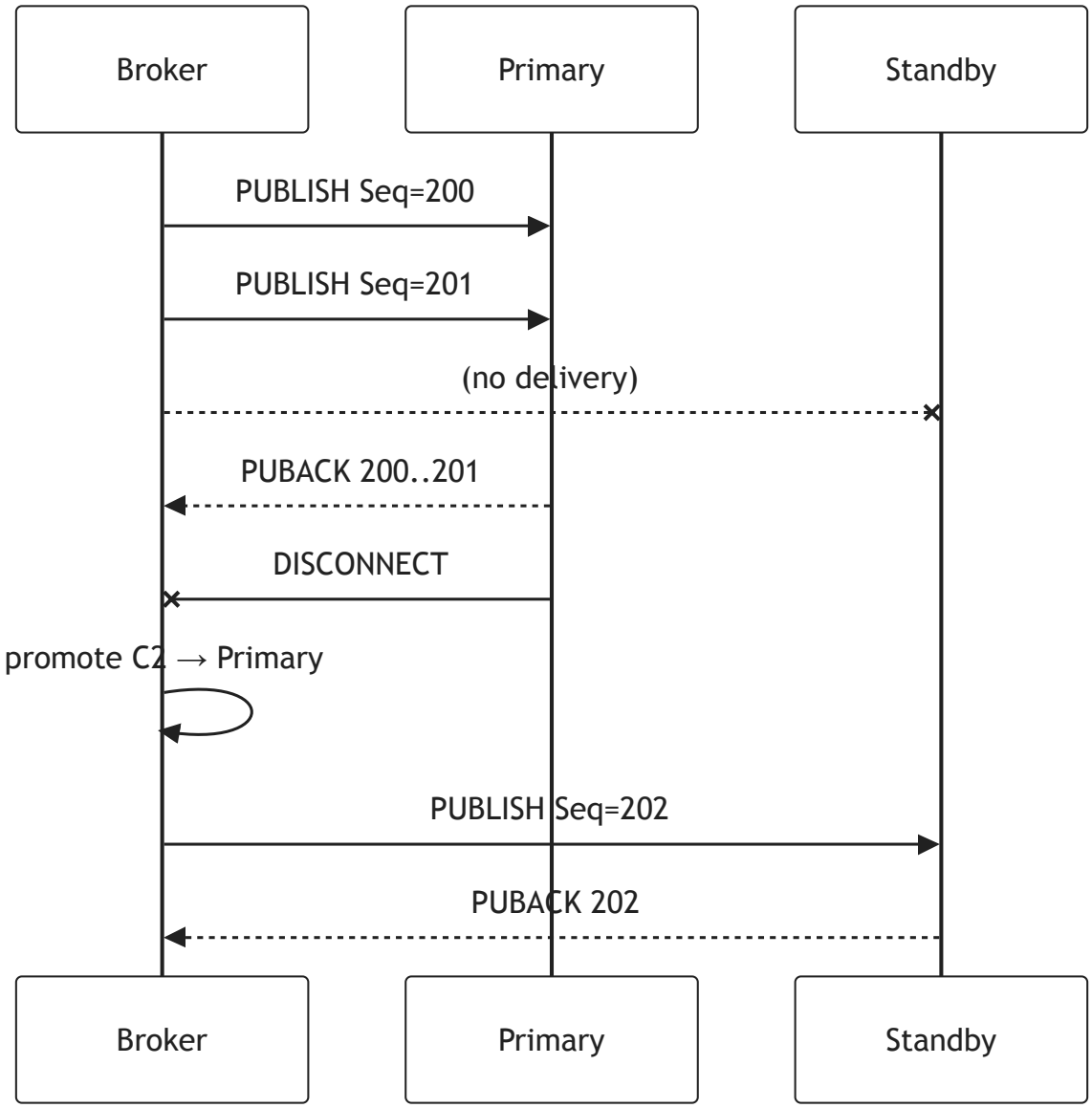

Diagram 10

7. Gap Detection + Exactly-Once

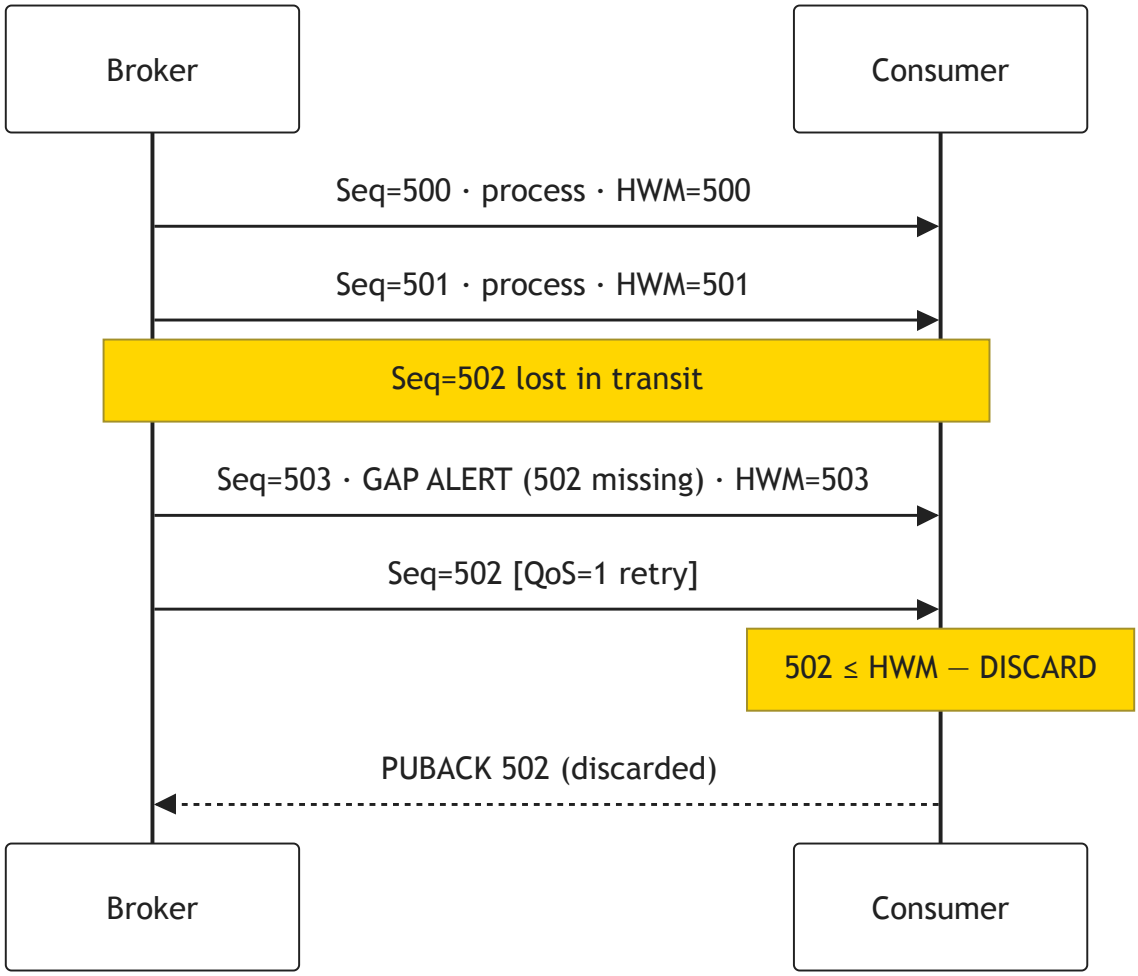

Diagram 11

8. Epoch Reset on Cluster Failover

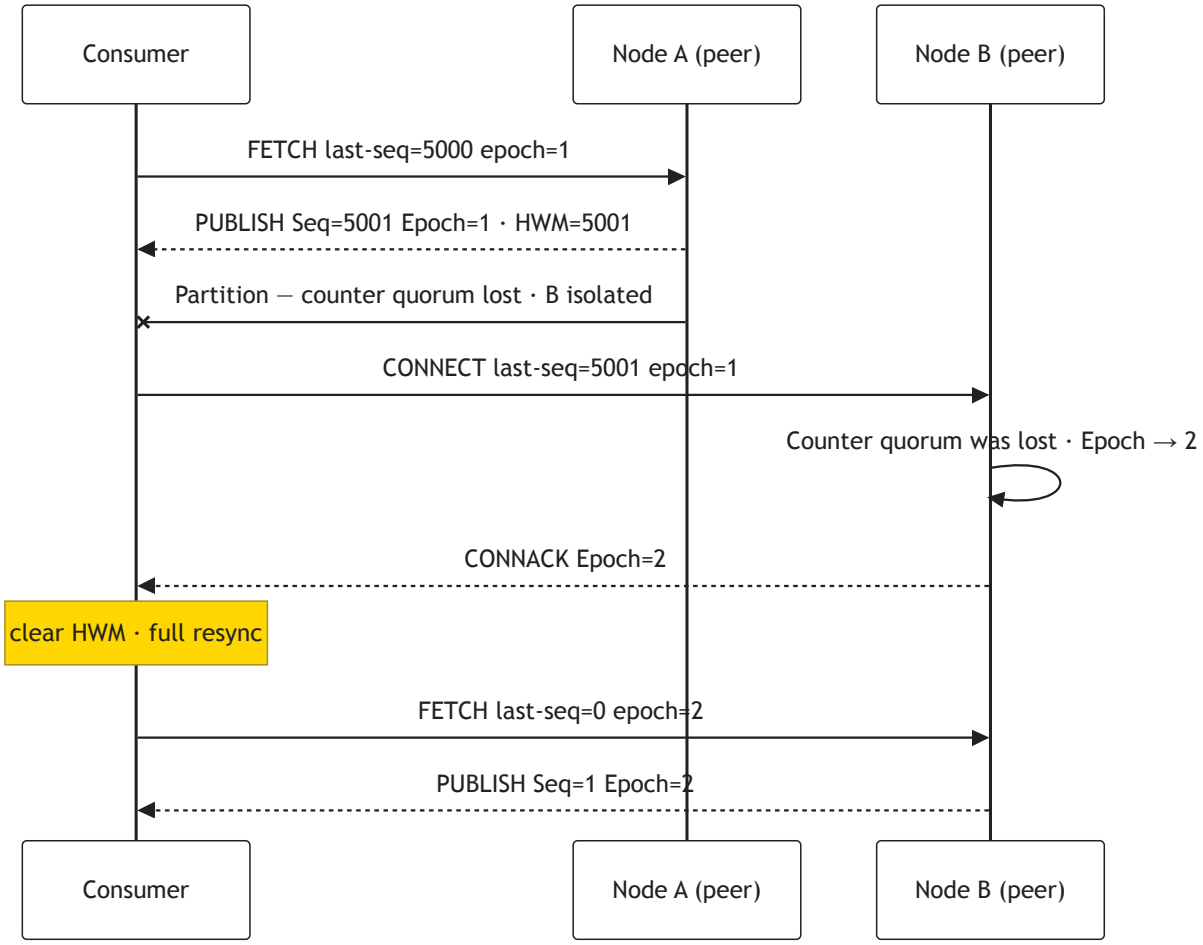

Diagram 12

Cluster Failover & Epoch Management Diagrams

> ***Architecture note:*** *HiveMQ is a* ***masterless, shared-nothing cluster*** *— all nodes are peers with no permanent leaders and no consensus protocol (Raft/Paxos). Sequence monotonicity is guaranteed by a* ***Distributed Sequence Counter*** *(atomic CAS) in HiveMQ's replication data grid, not by sticky routing to a leader node.*

1. Masterless Cluster — Any Node Sequences Any Publish (C4)

HiveMQ v6.0 — Masterless Cluster (all nodes are peers)

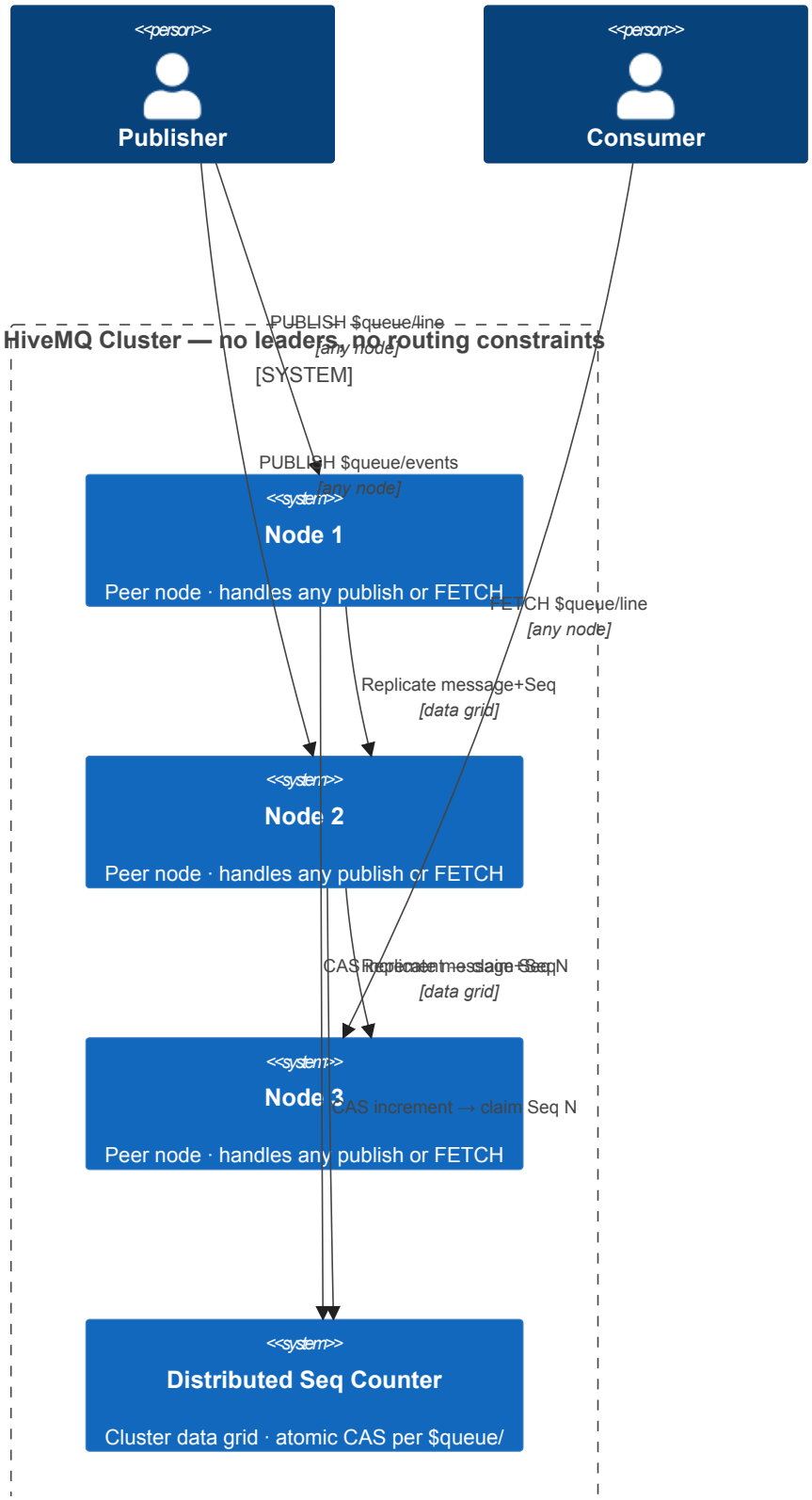

Diagram 13

2. Sequence Assignment — Distributed Counter

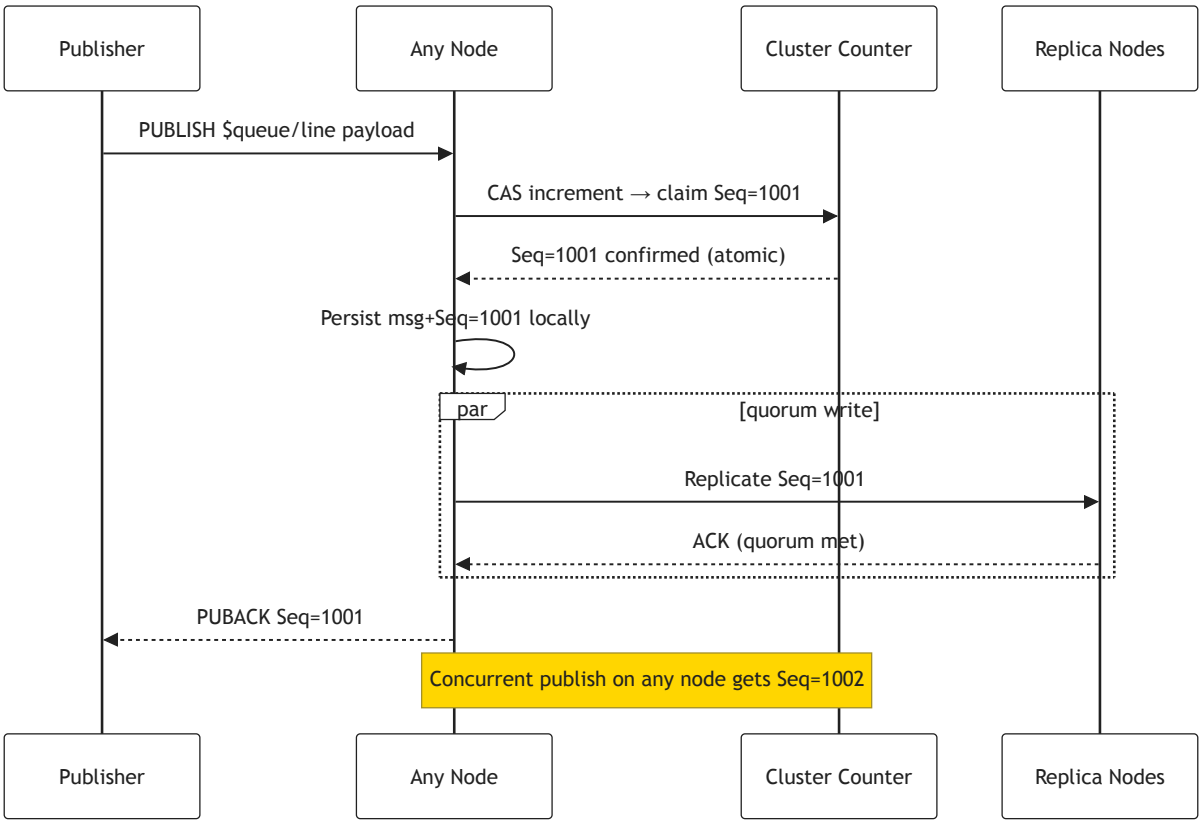

Diagram 14

3. Node Crash — No Election Required

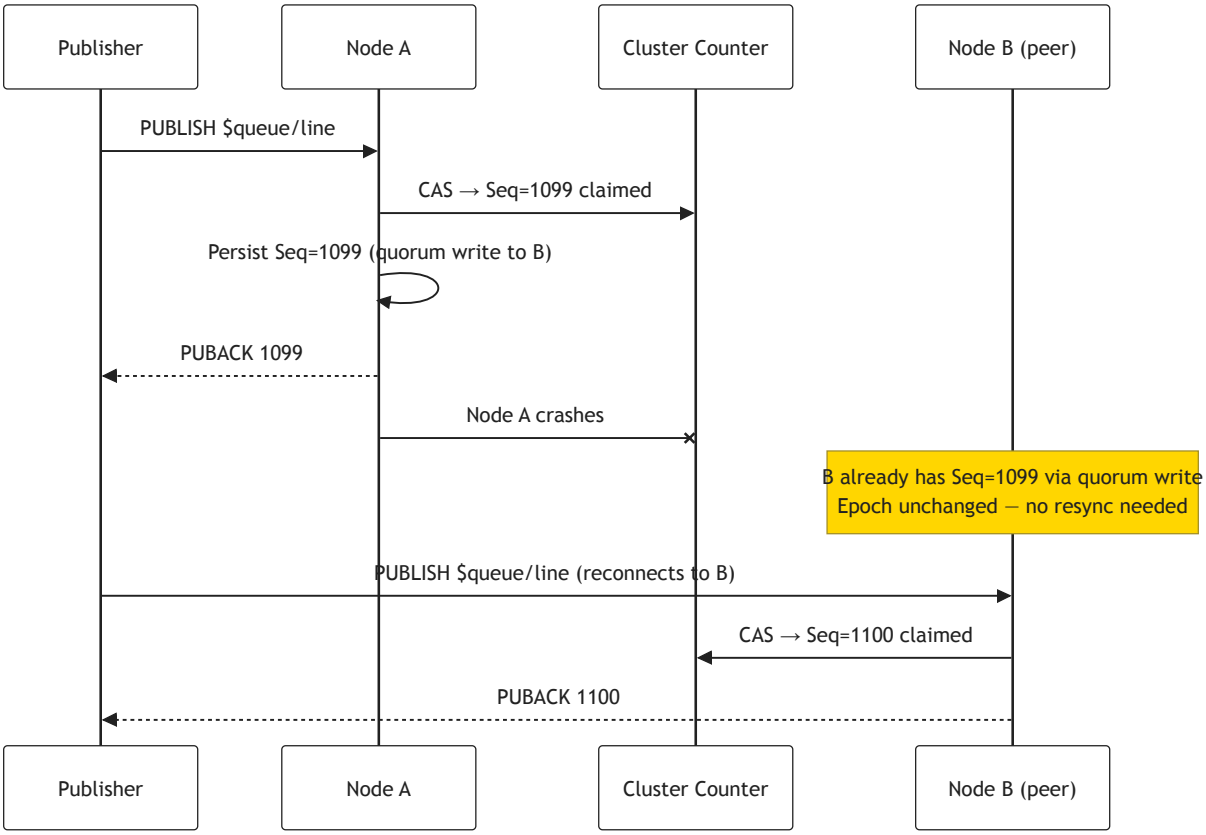

Diagram 15

4. Partition Event — When Epoch Increments

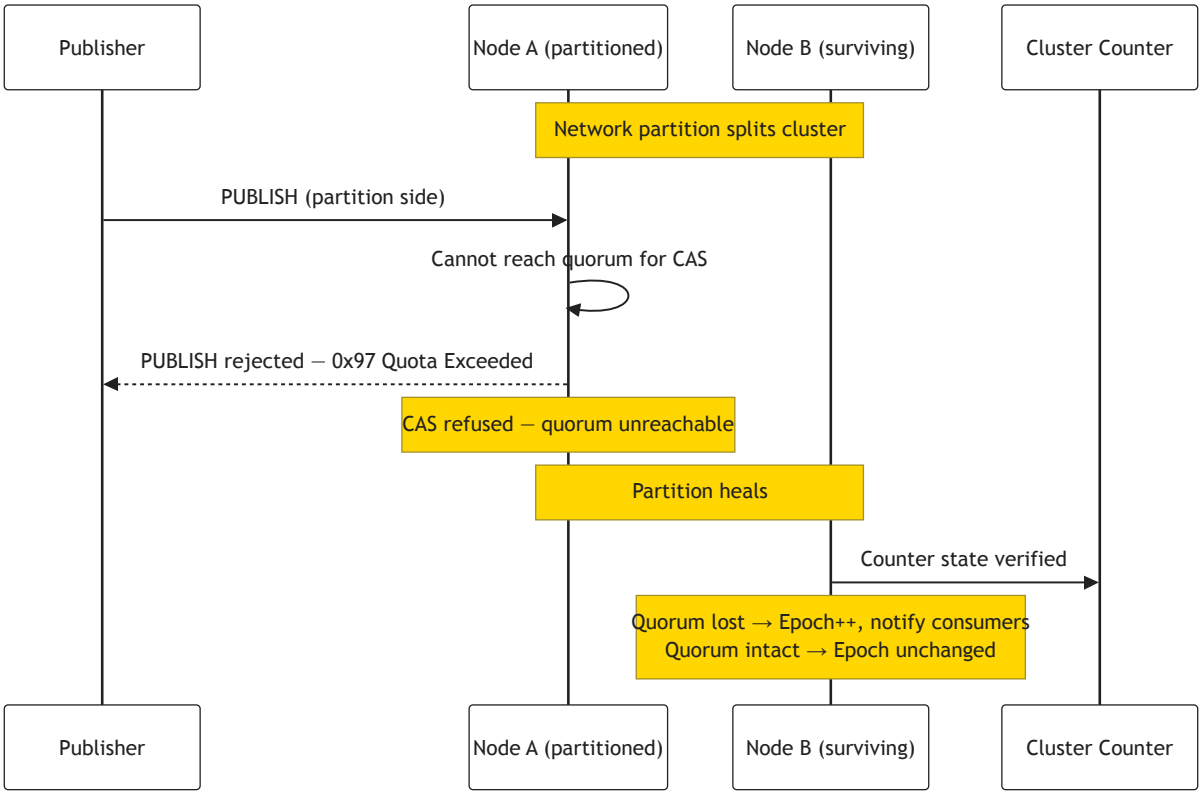

Diagram 16

5. Failure Outcomes Decision Tree

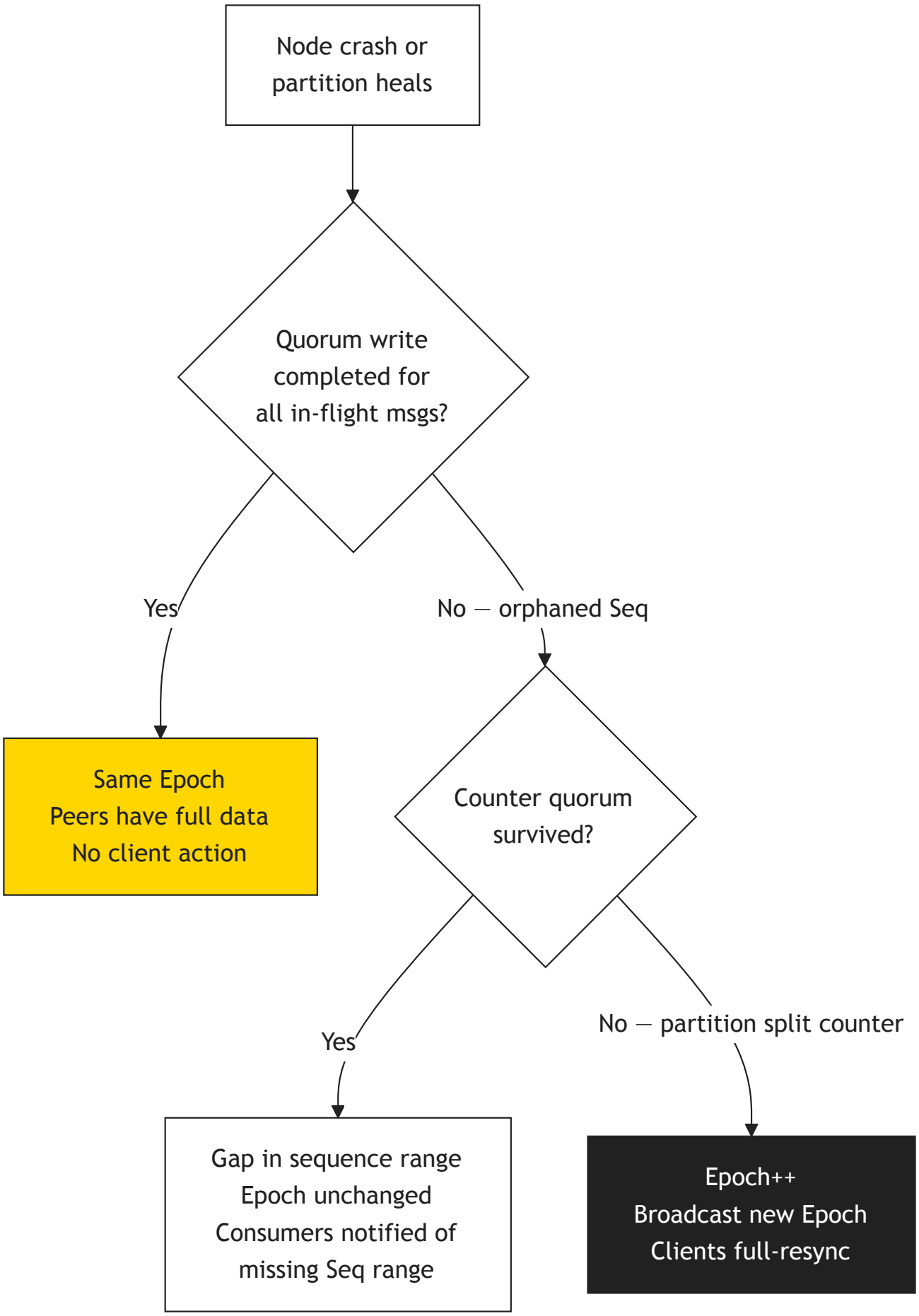

Diagram 17

6. Client Reconnect Decision Tree

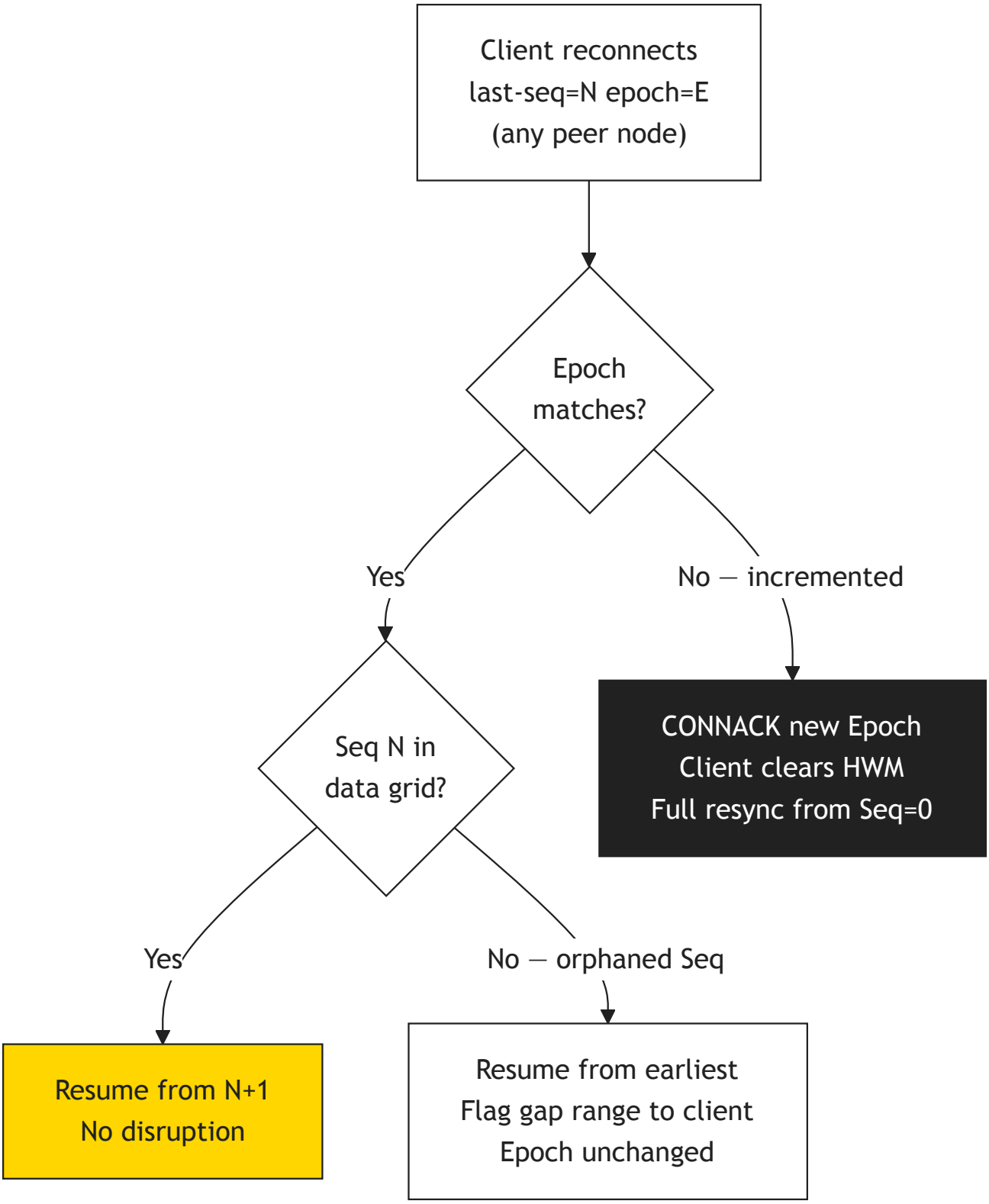

Diagram 18

7. Tiered Storage

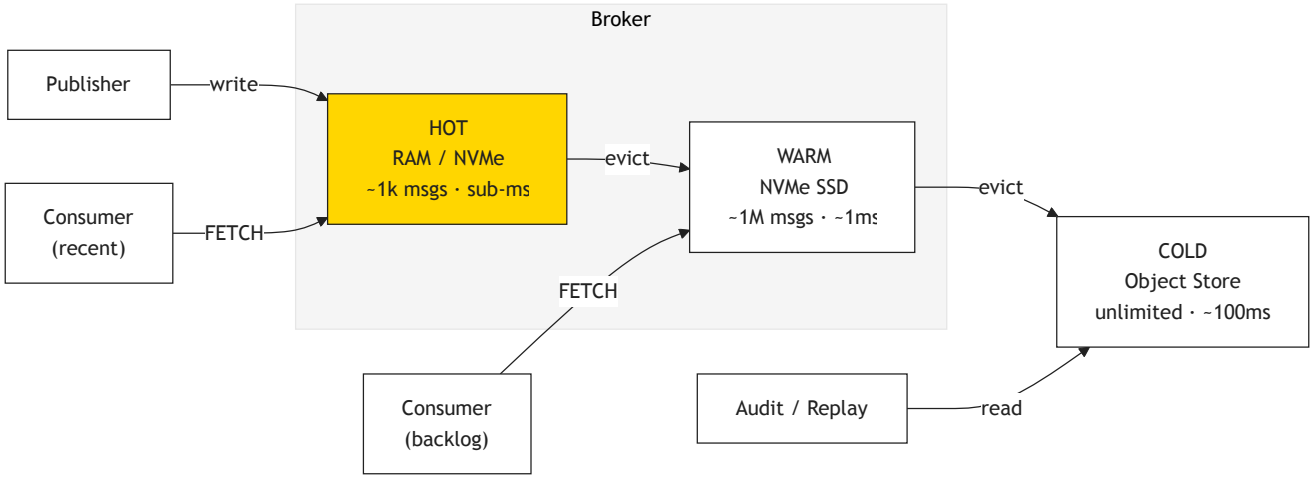

Diagram 19

8. Epoch Timeline

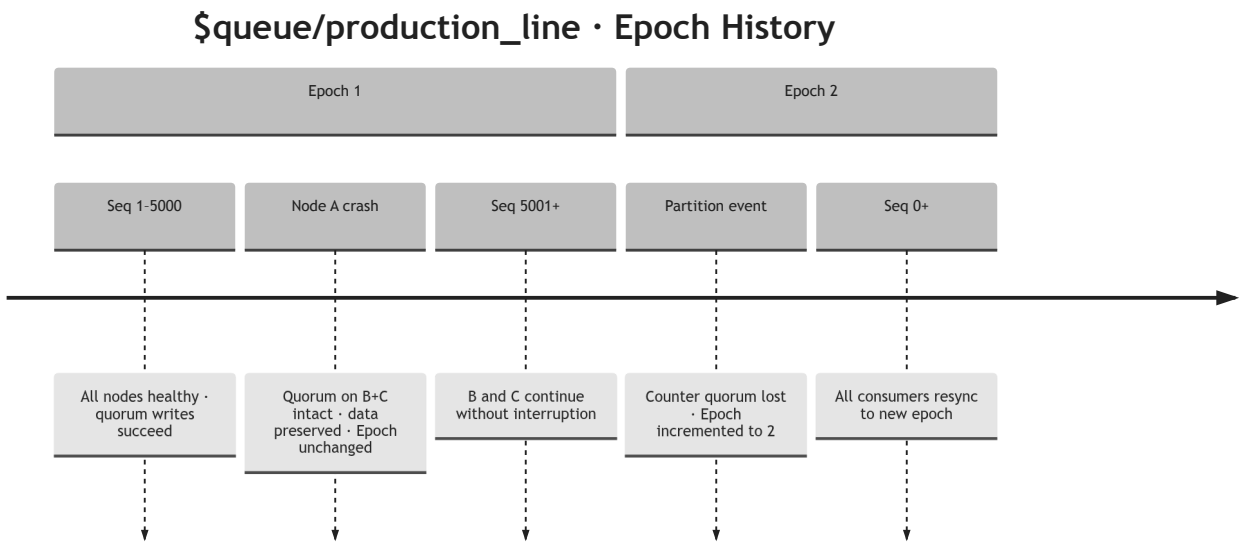

Diagram 20

Critical Analyses of MQTT v6.0

> *Three independent perspectives critiquing the MQTT v6.0 proposal.*

Part 1 — Security Analyst Perspective

Authored from the perspective of an industrial cybersecurity engineer with SCADA/ICS background, focused on OT/IT convergence risks.

1.1 Executive Verdict

MQTT v6.0 introduces meaningful architectural improvements for industrial messaging but expands the attack surface in ways the proposal does not adequately address. **The proposal is not ready for security-conscious deployment without significant additions to the specification.** The gaps are not fatal, but they are non-trivial.

1.2 Threat Model Gaps

1.2.1 Sequence Number as an Oracle

The Stream Sequence Number (`0x30`) is a monotonically increasing, cluster-wide counter assigned to every `$queue/` message. Any authenticated client that can publish and read the PUBACK Seq value gains **exact knowledge of global queue throughput**, message volume, and burst patterns — without consuming a single message.

A threat actor with read-only access to `$queue/scada/events` can infer: - Production rates across a facility - Fault event frequency and duration - Whether a process halt has occurred (Seq stops incrementing)

The proposal does not classify Seq values as sensitive metadata, does not propose per-consumer Seq obfuscation, and does not restrict Seq visibility to consumers. For regulated industries (NERC CIP, IEC 62443), publishing operational telemetry rates to all authenticated clients via a side-channel is a material information disclosure risk.

Recommendation: The spec should require that PUBACK Seq values are visible only to the publishing client, not leaked to other clients via CONNACK negotiation or shared subscription acknowledgements. Brokers should support `Seq-Blind` consumer mode where Seq is replaced with a client-local opaque token.

1.2.2 Epoch Reset as a Denial-of-Service Vector

Section 4.3.2 specifies that an Epoch increment forces all consumers to clear their High-Watermark and perform a full resync. The Epoch value is carried in CONNACK and PUBLISH packets — it is visible to any client that can connect to the broker.

A threat actor who can induce a cluster partition — or who can forge a CONNACK packet on a weakly-secured network segment (common in OT brownfield) — can trigger mass resync across all consumers, effectively causing a **denial of service on the consumer tier** for the duration of the resync window.

The proposal's mitigation (`v6-resync-jitter-ms`) addresses the thundering herd from legitimate partitions but does not address adversarial Epoch injection.

Recommendation: Epoch values in CONNACK MUST be authenticated via a broker-signed MAC or HMAC-SHA256, allowing clients to detect forged Epoch increments. This requires TLS mutual authentication (mTLS) to be normative for `$queue/` operations, not advisory.

1.2.3 `$SYS/queues/*/fetch` as an Unauthenticated Control Plane

The Virtual FETCH mechanism tunnels FETCH requests through `PUBLISH $SYS/queues/{name}/fetch`. In the compat-mode deployment, the broker extension reads this topic to release message batches.

The proposal does not specify authorization rules for `$SYS/queues/` writes. In most MQTT deployments, `$SYS/` is read-only for clients by convention. However, the proposal requires clients to *publish* to this namespace, which reverses the convention and may bypass ACL rules that blanket-deny `$SYS/` writes to non-admin clients.

A malicious client that can publish to `$SYS/queues/targetqueue/fetch` with `last-seq=0` can force the broker to stream the entire queue history to any consumer group — a data exfiltration path through the broker's own FETCH mechanism.

Recommendation: `$SYS/queues/*/fetch` MUST be protected by per-client, per-queue ACL identical to the `$queue/` subscription ACL. The spec must state this explicitly rather than deferring to broker implementation.

1.2.4 HWM Manipulation via Message Replay

The exactly-once deduplication relies entirely on the client's in-memory High-Watermark. The spec states that `Seq ≤ HWM → PUBACK but discard`. HWM is not persisted to durable storage in the base spec — it is reconstructed on reconnect.

If a client reconnects after a crash and the broker is still holding in-flight Seq values below the new HWM (derived from the last acknowledged FETCH), the client may re-process messages it already handled, violating exactly-once semantics. The gap between "last Seq the client persisted to its own DB" and "last Seq the broker tracks as committed" is an integrity risk in safety-critical command pipelines.

For `$queue/safety/interlocks` carrying process start/stop commands, duplicate processing of a "process start" command is a physical safety issue, not merely a data integrity issue.

Recommendation: Exactly-once guarantees for QoS=1 MUST require broker-side, per-consumer-group committed-HWM persistence (described in performance-analysis.md section 2.2) as normative, not optional. The base spec should not permit a compliant broker to hold HWM only in memory.

1.3 Missing Security Primitives

Gap	Risk Level	Mitigation Required
No mandatory mTLS for `$queue/`	High	Normative mTLS requirement in §4
No Seq visibility access control	Medium	Per-consumer Seq masking option
No Epoch authentication	High	Broker-signed Epoch in CONNACK
`$SYS/queues/` write ACL undefined	High	Explicit ACL requirement in compat spec
HWM not persisted by default	Medium–High	Normative broker-side HWM persistence
No message integrity field	Medium	Optional HMAC User Property recommendation
No audit trail for SQMC group membership changes	Medium	Mandatory SQMC group change event to `$SYS/`

1.4 Positive Security Properties

To be balanced: the proposal does introduce features that *improve* security posture relative to v5.0:

- **Named durable queues** make message provenance auditable (queue name + Seq = immutable event ID for 90-day regulatory retention).
- **Competing consumer mode** eliminates shared subscription race conditions that can cause message loss in QoS=1 — a correctness improvement that benefits audit trails.
- **Epoch tracking** provides a tamper-evident marker: if the Epoch advances unexpectedly, consumers can detect cluster disruption that might indicate an infrastructure attack.

The framework is sound. The spec needs hardening before it is appropriate for IEC 62443 Security Level 2+ environments.

Part 2 — OT SRE Perspective

Authored from the perspective of a senior site reliability engineer responsible for operational technology infrastructure at a large industrial facility — substations, PLCs, DCS, SCADA, and the middleware connecting them.

2.1 Executive Verdict

MQTT v6.0 solves the right problems but creates new operational burdens that the proposal underestimates. **The migration cost for existing OT deployments is significantly higher than the proposal acknowledges.** For brownfield facilities running MQTT v3.1.1 at the edge, v6.0 is a multi-year programme, not a phased upgrade.

2.2 Operational Concerns

2.2.1 The “Two Tracks” Problem Creates Indefinite Parallel Operations

The proposal defines Track A (native v6.0) and Track B (compat extension), recommending “Native v6.0 with an optional Compatibility Layer for hybrid deployments.” In practice, industrial facilities do not retire protocol versions on a planned schedule — brownfield devices stay in service for 10–25 years.

What this means in the field: An OT team will run Track B (compat) indefinitely, not as a transition, but as a permanent parallel stack. The compat layer then becomes load-bearing infrastructure that must be maintained across HiveMQ major versions.

Track B is not a “transition path” — it is a *permanent architecture tier* for any facility that cannot replace embedded MQTT clients on PLCs, RTUs, or smart meters. The proposal should acknowledge this and commit to long-term compat layer support with explicit SLA guarantees.

2.2.2 Epoch Reset Operational Impact is Unquantified

Section 4.3.2 specifies that an Epoch reset requires all consumers to perform a full resync from Seq=0. For a `$queue/` with 90 days of regulatory data at 500k msg/sec:

```
90 days × 86,400 sec/day × 500,000 msg/sec = ~3.9 × 10^12 messages
```

A full resync of 3.9 trillion messages is not a recovery operation — it is a service restoration incident measured in **days to weeks**, not minutes. The proposal's staggered resync jitter (`v6-resync-jitter-ms`) prevents the thundering herd but does not address the total volume of data that consumers must re-read.

The proposal needs: 1. A normative upper bound on how far back a "full resync" actually reads (e.g., "from the last `committed_hwm` , not from Seq=0"). 2. A mechanism for consumers to declare their own recovery horizon (e.g., "I only need messages from the last 24 hours for real-time dashboards"). 3. Documented broker-side resync budgeting — how many concurrent resyncs can a broker support without degrading live traffic?

2.2.3 `$stream/` and `$log/` Durability Tiers Require New Operational Runbooks

The performance analysis introduces `$stream/` (async) and `$log/` (eventual) namespaces. From an OT operations perspective, these three namespaces have fundamentally different failure modes:

- `$queue/` : Message is either committed or rejected. No ambiguity.
- `$stream/` : Message appears committed (PUBACK issued) but may be lost on node failure before replication. Ops team must assume messages are "probably there."
- `$log/` : PUBACK on local write, eventual consistency, gaps are possible. Ops team must build gap-tolerant consumers for every `$log/` subscriber.

OT operators who are accustomed to a single namespace (`$queue/`) with deterministic guarantees now have three namespaces with three different reliability contracts. **Misconfiguring a safety-critical sensor stream from `$queue/` to `$stream/` is a silent reliability regression** — the system continues to operate normally until a node fails, at which point data loss occurs with no alarm.

Recommendation: The spec should mandate a mandatory `Durability-Confirmed` property in CONNACK for any client subscribing to `$stream/` or `$log/` , explicitly informing the client of the durability tier. Brokers should refuse `$stream/` subscriptions from clients that have not acknowledged the reduced durability guarantee.

2.2.4 Adaptive Batch Size Tuning Is Not Observable

The `v6-batch-target-ms` property asks the broker to calculate batch size based on "queue throughput." There is no mechanism in the proposal for a consumer to inspect what batch size the broker actually chose, nor to observe the broker's throughput estimate.

In OT environments, when message processing falls behind (a common condition during fault storms), the ops team needs to answer: "Is the broker delivering batches correctly, or is our consumer too slow?" Without visibility into the broker's batch-size calculation, this is a black box diagnostic problem.

Recommendation: Add a `v6-batch-actual` User Property to FETCH response PUBLISH packets, reporting the actual batch size used and the broker's throughput estimate at time of dispatch.

2.2.5 SQMC Group Membership Has No Health Signal

The SQMC competing consumer model tracks group membership at the broker. If a consumer silently stalls (network partition at the consumer, not the broker), the broker may continue dispatching messages to the stalled consumer's in-flight lock set indefinitely — blocking the `committed_hwm` from advancing for the entire group.

The proposal does not specify: - A per-consumer in-flight timeout (how long can a lock be held before the broker considers the consumer failed and re-dispatches?) - A SQMC health status endpoint or `$SYS/` topic for group membership monitoring - Alerting or operator notification when `committed_hwm` stalls

In a production SCADA environment, a stalled HWM is a high-severity incident. It must be observable.

2.3 Deployment Readiness Assessment

Scenario	v6.0 Readiness	Concern
Greenfield broker + new OT devices	Ready	Ideal target environment
Greenfield broker + legacy MQTT v3 devices	Partial	Track B required indefinitely
Existing HiveMQ + v5 clients, upgrade broker	Partial	Extension plugin needed; Epoch semantics change
Multi-vendor broker environment	Not ready	SQMC and Epoch semantics are HiveMQ-specific
Air-gapped OT network, infrequent updates	Not ready	HWM persistence and Epoch reset recovery not defined for offline operation

---

2.4 What OT SREs Actually Need

The proposal solves the engineering problems well. What is missing is the **operational layer**:

1. **Health and observability APIs** — SQMC group health, HWM progress, resync status, broker-side batch stats, all exposed via `$SYS/` topics or a new `$MGMT/` namespace.
2. **Operational runbooks** for Epoch reset recovery — including expected duration at various queue depths.
3. **Durability tier migration tooling** — safe procedure for moving a topic from `$queue/` to `$stream/` without data loss.
4. **Long-term compat layer commitment** — explicit version support matrix for Track B.

Part 3 — Arlen Nipper Perspective

Authored from the perspective of Arlen Nipper, co-inventor of MQTT, arguing against the adoption of MQTT v6.0.

Note: The views expressed here are a constructed argument representing this perspective for design evaluation purposes. They do not represent statements made by Arlen Nipper.

3.1 Opening Statement

I have spent thirty years watching MQTT succeed precisely because it does one thing exceptionally well: it moves small messages reliably over unreliable networks with minimal overhead. The protocol fits in a 128KB microcontroller. It runs over satellite links at 9600 baud. It works in a refinery where the network engineer left the company in 2011 and no one knows what hardware is in the ceiling.

MQTT v6.0 proposes to solve problems that are real, but the solutions it proposes are not MQTT solutions. They are message queue solutions borrowed from AMQP and Kafka and dressed in MQTT clothing. The result is a protocol that will be too complex for the constrained devices MQTT was designed to serve, while being less capable than the systems it is trying to imitate.

3.2 The Core Argument: Protocol Creep Destroys Interoperability

MQTT's greatest strength is that every compliant broker and client is interoperable. A Mosquitto broker, a HiveMQ cluster, a tiny RTOS client on an ESP32, and an enterprise application on AWS IoT Core all speak the same language. You can replace any component without changing any other.

MQTT v6.0 breaks this in two ways.

First, it introduces a new packet type (FETCH, Type 16) that is not backward-compatible. Any broker that does not support FETCH cannot serve v6.0 consumers. Any v6.0 consumer that requires FETCH cannot fall back to a v5.0 broker. The Track B compat layer is explicitly a workaround for this fragmentation, not a solution.

Second, it introduces broker-side semantics — SQMC group membership, distributed sequence counters, sliding window HWMs — that are not part of the wire protocol but are required for correct operation. Two brokers that both claim MQTT v6.0 compliance may behave entirely differently when a consumer crashes mid-batch, because the spec leaves the in-flight timeout undefined. This is not interoperability. This is vendor lock-in with a standardised handshake.

3.3 The Right Tool Exists

Every feature in MQTT v6.0 already exists in a mature, production-proven system. The proposal is asking MQTT to become a subset of these systems:

v6.0 Feature	Existing Solution
Named durable queues	RabbitMQ AMQP queues, since 2007
Pull-based FETCH with batch control	Apache Kafka consumer poll API, since 2011
Competing consumer with exactly-once	AMQP 1.0 link credits + settlement modes
Sequence numbering + gap detection	Kafka topic partition offsets
Cluster-wide atomic sequence counter	Zookeeper / etcd distributed counter
Epoch-based failover signalling	Kafka group coordinator epoch

The industrial customers who need these features should be using Kafka, RabbitMQ, or AMQP 1.0 at the tier where these features matter — the data historian, the analytics pipeline, the command-and-control system. These are server-to-server or server-to-cloud links where running a Kafka client is entirely feasible.

At the edge — the RTU, the PLC, the smart meter — MQTT v5.0 is entirely sufficient. These devices do not need FETCH semantics. They need to publish telemetry and receive commands. MQTT v5.0 does this well.

3.4 The MQTT Gateway Pattern Is the Correct Architecture

The proposal acknowledges that many edge devices will remain on MQTT v3.1.1 or v5.0 indefinitely. It responds by creating a compat layer. I would respond differently: **use a gateway**.

```
[Edge Device MQTT v5.0] → [Edge Broker (HiveMQ/Mosquitto)]
                                    ↓
                            [Protocol Bridge / Gateway]
                                    ↓
                    [Kafka / RabbitMQ / AMQP 1.0 — full semantics]
                                    ↓
                            [Analytics / SCADA / Cloud]
```

This architecture: - Keeps edge devices on the protocol they already run - Provides full queue semantics at the tier that needs them - Does not require edge firmware updates - Does not introduce new MQTT broker implementations - Uses battle-tested systems for the queue layer

The proposal essentially reinvents this architecture within a single protocol. That is not simplification — it is complexity concealment.

3.5 The Constrained Device Problem

Section 3.1 of the proposal states that "all three tiers (`$queue/` , `$stream/` , `$log/`) use the same FETCH, Seq, and Epoch semantics." A compliant v6.0 client must:

1. Track a High-Watermark (or sliding window HWM) persistently across power cycles
2. Detect Epoch changes and discard HWM on Epoch mismatch
3. Issue FETCH packets (or Virtual FETCH via `$SYS/`) with correct `last-seq` values
4. Handle gap alerts (Seq N missing, resume from N+1)
5. Implement exactly-once deduplication via HWM comparison

This is not a protocol that fits in 128KB. This is not a protocol that a firmware engineer at a meter manufacturer can implement correctly in six months without dedicated protocol expertise. MQTT's original success came from the fact that a competent embedded developer could read the spec in a day and write a correct client in a week.

The reference Python shim in this proposal is 300 lines of non-trivial async code. That is the minimum viable client. The Java extension is longer. The constrained device story is "use the compat layer and pretend it's MQTT" — which is precisely what the gateway pattern achieves without protocol fragmentation.

3.6 A Constructive Alternative

I am not arguing that HiveMQ should not build these features. I am arguing that these features should not be called MQTT v6.0, and should not require an OASIS standards process that will take years, produce inevitable compromises, and lock in design decisions that may not survive first contact with production deployments.

What I would recommend instead:

1. **HiveMQ Extension v2 for Advanced Queuing** — implement `$queue/` , SQMC, sequence numbers, and FETCH as a first-class HiveMQ extension, not a protocol revision. Ship it. Let the market validate the semantics. Allow competitors to implement compatible extensions.

2. **An OASIS MQTT Best Current Practice (BCP) document** — define the User Property schema (`v6-seq` , `v6-epoch` , `v6-semantics`) as a standardised extension pattern. This enables interoperability without a wire protocol break.

3. **Five years of production data** — if the HiveMQ extension proves out at DEP scale, with demonstrated adoption across multiple vendors, then bring it to OASIS as a v6.0 proposal with real-world validation behind it.

MQTT became the dominant IoT protocol because it was simple, stable, and widely implemented. Those properties are worth more than any individual feature. The right way to honour that legacy is not to add features — it is to build the features where they belong, at the system layer, and let MQTT remain what it has always been: a lightweight, reliable, interoperable publish-subscribe transport.

Summary Comparison

Dimension	Security Analyst	OT SRE	Arlen Nipper
Overall verdict	Needs hardening	Migration cost underestimated	Wrong layer for these features
Top concern	Seq as info oracle, Epoch injection	Epoch reset at scale, compat permanence	Protocol fragmentation, constrained device burden
Fatal flaw?	No — addressable	No — operational gaps	Yes — misplaced complexity
Would adopt?	Yes, with security addendum	Yes, greenfield only	No — gateway pattern preferred
Primary ask	Normative mTLS + Seq ACL + Epoch MAC	Observability APIs + resync runbooks	Ship as HiveMQ extension first